Optimizer Dynamics, Schedules and Batch Size for Llama-3.2-Style Pretraining

Status report

October 20, 2026

Abstract

We ask whether training-dynamics measurements, i.e. gradient and update statistics logged during training, can guide optimizer and schedule decisions for Llama-3.2-style language models instead of blind sweeps. On a 174M-parameter proxy and a 1B-parameter main model we compare Muon, NorMuon, SOAP, AdamW and Sophia, test per-block optimizer settings, an optimizer switch, decay length and shape, and batch-size schedules. The optimizer family dominates: Muon, NorMuon and SOAP tie, while AdamW and Sophia trail, and no per-block change beats seed noise. Decay branches show that switching from NorMuon to AdamW never pays. A longer learning-rate decay is the largest gain we found. Batch schedules interact with training progress: a batch increase acts like a partial decay at 1B tokens, while a larger batch late in training helps at 3B tokens. Dynamics metrics such as weight rank and oscillation are reliable fingerprints of the optimizer family but do not predict loss within a family. The two-batch sharpness estimate and the gradient noise scale are both misleading in this setting.

Contents

1 Purpose **3**
1.1 TL;DR . . . 3

2 Models and baselines **4**
2.1 Proxy model . . . 4
2.2 Main model . . . 4
2.3 Optimizers . . . 5
2.4 Noise level and decision rule . . . 5

3 How we experimented **7**

4 Metric definitions **8**

5 Which optimizer? (proxy and main models) **10**

6 What the dynamics metrics show **11**
6.1 Oscillation . . . 11
6.2 Preconditioning: how far the update departs from the gradient . . . 12
6.3 Overshoot . . . 14
6.4 Rank, and how rank relates to final loss . . . 15
6.5 Norm layers: update speed is set by LR $\times$ WD . . . 17
6.6 The rise in sharpness during the decay is a measurement artifact . . . 19
6.7 Weight-decay timescale . . . 19

7 Experiment: per-block changes to NorMuon (proxy-1B) 20

8 Experiment: switch from NorMuon to AdamW mid-run? (proxy-6B) 21

9 Experiment: decay length and shape (main model, 1B tokens) 22

10 Experiment: batch size and batch schedule (main model) 23
10.1 At 1B tokens 23
10.2 At 3B tokens: a second doubling 24
10.3 Gradient noise scale 26

11 Which measurements helped which decisions 28

12 Limitations 28

A W&B run names 32

B How the settings were tuned 36

C Hyperparameter sweeps that set the baselines 37

D Logger validation 38

1 Purpose

This project asks whether *training-dynamics measurements* can guide optimizer and schedule decisions. The approach follows the warmup–stable–decay (WSD) analysis of Hu et al. (2024), who tracked gradient norms, consecutive-gradient cosines and directional curvature during training, and the update-spectrum monitors of Huang et al. (2026). Concretely, we wanted to know:

1. which optimizer family to use, and whether different blocks (attention, MLP, embedding, norms) should get different optimizers or settings;
2. whether to switch optimizer partway through training (for example Muon early, AdamW late);
3. how long and what shape the learning-rate decay should be;
4. what batch size and batch-size schedule to use.

For each question we ran controlled variants of a baseline, logged the dynamics, and checked which measurements predicted the final loss.

1.1 TL;DR

1. **The optimizer family decides the final loss; per-block tuning inside a family does not.**
   - Muon, NorMuon and SOAP tie on the proxy model. AdamW is 0.03–0.10 worse, and Sophia is worse again. This agrees with the benchmark of Wen et al. (2025).
   - Matrix-based optimization methods like Muon family and SOAP generally outperform vector-based methods like AdamW and Sophia on the proxy model (174M parameter) and main model (1B parameter).
   - Head-to-head on the main model, NorMuon leads SOAP by 0.015 at 1B tokens, while SOAP leads by 0.006 at 3B tokens (single seeds). NorMuon consistently gains more in the LR decay (+0.014 to +0.029), whereas SOAP leads during the stable phase.
   - None of the five per-block modifications of NorMuon beat the baseline by more than seed noise, on either the proxy or the main model at 3B tokens. Orthogonalizing Q, K, V per attention head is the nominally best variant on both models (-0.001 to -0.004), but the gain remains within seed noise (± 0.004).
2. **A longer learning-rate decay is the largest single gain found.** On the main model, decaying over the last 30% of steps instead of 10% improves NorMuon by 0.030 (2-run means). Linear and $1-\text{sqrt}$ decay shapes perform the same (Section 9).
3. **The best batch schedule depends on how far training has progressed** (Section 10).
   - In a 1B-token run, increasing the batch acts like a partial learning-rate decay. Raising the LR when the batch grew hurt, and so did running the final decay at a larger batch.
   - In a 3B-token run, a larger batch late in training helped. Doubling to 128 sequences at 1.5B tokens and decaying at 128–256 improved the final loss by 0.020.
   - This fits a "useful batch grows with progress" picture.
4. **Dynamics metrics are reliable optimizer fingerprints but poor predictors within a family** (Section 6).
   - *Rank:* weight and update rank track final loss *across* optimizers (Spearman $\rho = -0.95$) but not *within* one optimizer.

- *Oscillation:* Muon-family optimizers oscillate step to step; SOAP, AdamW and Sophia do not. The oscillation grows when batch noise drops, so it comes from step size and curvature, not noise.
- *Overshoot and consistent head differences:* every optimizer overshoots the line minimum on every step, i.e. it trains at the edge of stability. Across all optimizers and both models (Table 10), attention heads show consistent differences: `attn_v` overshoots most ($\text{ov}_n \approx 0.13$–0.50) and has the lowest noise scale ($\approx$ 0.3–0.4M tokens), followed by `attn_k` and `attn_o`, while `attn_q` overshoots least.

5. **Two common measurements are misleading here.**
   - The two-batch sharpness estimate (Hu et al., 2024) rises during the decay only because of a $1/\text{LR}$ noise bias.
   - The gradient-noise-scale estimate of the critical batch (McCandlish et al., 2018) places the first batch doubling correctly at 1B tokens. Beyond that it fails: at 3B tokens it is flat ($\approx$0.4–0.6M tokens), it *falls* when the batch is raised, and in the decay it jumps for reasons unrelated to batch size. It cannot time later batch increases, consistent with Merrill et al. (2025).
6. **Findings in this report may not generalize**
   - The models in experiments are in very early stage of training. The amount of training tokens is far below common practices for large-scale language model training.
   - See Section 12 for a discussion of the limitations and potential generalization issues of the findings in this report.

2 Models and baselines

2.1 Proxy model

Table 1: Proxy-model setup.

Item	Setting
Architecture	Llama-3.2 style (Grattafiori et al., 2024), 174M parameters: hidden 768, 12 layers, 12 query heads / 4 KV heads, head dimension 64, MLP 2048, tied input/output embedding, vocabulary 128,256
Sequence length	2,048
Short budget ("proxy-1B")	1B tokens, batch 128 sequences (262k tokens/step), 3,814 steps
Long budget ("proxy-6B")	6B tokens, batch 256 sequences (524k tokens/step), 11,444 steps
LR schedule	WSD (Hu et al., 2024; Hägele et al., 2024): 2% warmup, constant LR, linear decay to 0 over the last 10% of steps
Weight decay	0.1, decoupled (Loshchilov and Hutter, 2019); norm-gain WD depends on the optimizer (Table 3)

2.2 Main model

Both models are trained on the same Dolma 3 data, tokenized with the Llama 3 tokenizer.

Table 2: Main-model setup.

Item	Setting
Architecture	Llama-3.2-1B (Grattafiori et al., 2024), sequence length 8,192
Batch (baseline)	32 sequences (262k tokens) for the first 50% of tokens, then 64 (524k tokens)
Budget	1B tokens (2,860 steps); a 3B-token budget is used for the batch study
LR schedule	WSD with 2% warmup and linear decay over the last 10% of steps
Optimizers	NorMuon (baseline), SOAP and AdamW (plus Muon and Sophia sweep baselines, Table 7)

2.3 Optimizers

- The baseline is NorMuon (Li et al., 2025), as implemented in the `muonium` library (muonium contributors, 2025).
  - Hidden 2-D weight matrices use NorMuon. Their updates are orthogonalized with the Polar Express iteration (Amsel et al., 2025), and their LR is scaled per matrix by $\sqrt{\max(1, \text{rows}/\text{cols})}$, as in the reference Muon implementation (Jordan et al., 2024).
  - The tied embedding and the norm gains use AdamW inside the same optimizer.
  - Muon (Jordan et al., 2024) is identical except that it lacks NorMuon's per-neuron normalization.
- SOAP (Vyas et al., 2025), AdamW (Loshchilov and Hutter, 2019) and Sophia (Liu et al., 2024) are each applied to all parameters.
- All hyperparameters are listed in Tables 3–5.
- Values marked † are `muonium` library defaults that the code relies on but does not log.
- Appendix B explains how each value was chosen.

Table 3: Training settings shared by every optimizer and both models.

Setting	Value
Gradient clipping	global norm 1.0
LR schedule	WSD in step units: linear warmup over 2% of steps, constant, linear decay to ≈0 over the last 10% of steps (variants in Sections 9–10)
Weight decay	decoupled; 2-D matrices and the tied embedding always decayed
Norm-gain (RMSNorm) weight decay	flag `-norm_wd`. Default: on for NorMuon/Muon (`muonium`'s AdamW group decays every parameter), off for SOAP, AdamW and Sophia (HF Trainer split). The SOAP and AdamW runs in this report turn it on unless noted (Table 24).
Data, packing	Dolma 3, Llama 3 tokenizer, document-masked packing

2.4 Noise level and decision rule

- **Proxy:** seed pairs differ by 0.001–0.004.
- **Main model, 30% decay:** seed pairs differ by 0.002–0.005.
- **Main model, 10% decay:** the spread is larger. Two runs with identical flags and seed ended at 2.9454 and 2.9324 (0.013 apart); training is not bitwise reproducible, and the short decay does not remove

Table 4: Optimizer hyperparameters, proxy model (174M, `train_scorer.py`). Batch 128 sequences (1B-token budget) or 256 (6B).

Optimizer	Peak LR	Betas, second moment, ε, WD	Norm-gain WD, specific
NorMuon / Muon, hidden matrices	0.01, $\times\sqrt{\max(1, r/c)}$	0.95, Nesterov†; second moment NorMuon: per-neuron EMA, $\beta_2 = 0.95^{\dagger}$, Muon: none; $\varepsilon = 10^{-8\dagger}$ (NorMuon); WD 0.1, cautious†	on; Polar Express, 5 steps†
NorMuon / Muon, embedding, norm gains	0.003	(0.9, 0.95); $\beta_2 = 0.95$; $\varepsilon = 10^{-8}$; WD 0.1	on; –
SOAP (all parameters)	0.004	(0.95, 0.95); $\beta_2 = 0.95$; $\varepsilon = 10^{-8}$; WD 0.1	on (off in some runs); official SOAP; preconditioner every 10 steps; max dim 10,000 (vocab dim not preconditioned); 1-D params get Adam
AdamW (all parameters)	0.001	(0.9, 0.95); $\beta_2 = 0.95$; $\varepsilon = 10^{-8}$; WD 0.1	off at 1B tokens, on at 6B; –
Sophia (all parameters)	2×10^{-4}	(0.965, 0.99); Hessian EMA, $\beta_2 = 0.99$; ε – (ratio clipped); WD 0.2 (old: 0.1)	off at 1B tokens, both at 6B; SophiaG; Hessian every 10 steps; $\rho = 0.05$ (old: 0.01)

Table 5: Optimizer hyperparameters, main model (Llama-3.2-1B). Only NorMuon, SOAP and AdamW were run in this report; Muon and Sophia rows give the tuned settings from earlier work for reference.

Optimizer	LR	Betas, ε, WD	Other
NorMuon + AdamW (baseline)	NorMuon 0.01 ($\times\sqrt{\max(1, r/c)}$); AdamW part 0.003	NorMuon momentum 0.95 Nesterov†, $\beta_2 = 0.95^{\dagger}$; AdamW part (0.9, 0.95), 10^{-8}; WD 0.1, cautious†	Polar Express 5 steps†; norm WD on
SOAP	0.003	(0.95, 0.95), 10^{-8}, WD 0.1	official SOAP; preconditioner every 10 steps; vocab dim not preconditioned; norm WD on
AdamW	3×10^{-4}	(0.9, 0.95), 10^{-8} in these runs (tuned with 10^{-5}; see Table 24), WD 0.1	norm WD on
Muon + AdamW (sweep baseline, Table 7)	Muon 0.01; AdamW part 3×10^{-4}	as NorMuon	0.5M fixed batch
Sophia (sweep baseline, Table 7)	5×10^{-5}	(0.965, 0.99), WD 0.2	$\rho = 0.05$; Hessian every 10 steps

the late-run divergence. For 10%-decay main-model comparisons we therefore use the mean of the two runs: NorMuon 2.9389, SOAP 2.9534.

- **Rule:** differences below 0.004 (proxy, main model with 30% decay) or 0.013 (main model with 10% decay) are treated as no effect unless they repeat across seeds. Single-seed results are labelled as such.

3 How we experimented

Our training scripts can be found in the repository at `https://github.com/yukicoder0509/training-llama-3-2`.

Instrumented training.

- Every run logs the metrics of Section 4 with our logger `optim_dynamics.py`.
- Metrics are logged for the whole model and for each block type (attn_q, attn_k, attn_v, attn_o, mlp_gate, mlp_up, mlp_down, embed, norm).
- Logging happens every 50 steps (every 25 in the 3B runs).
- The logger does not change training: runs are bit-identical with and without it.

Stable-phase summaries. Unless stated otherwise, a quoted metric is the median over the constant-LR part of training:

- proxy: from 30% of the way to the decay start, up to the decay start;
- main model: steps 500–1900, before the batch increase.

Controlled variants.

- Each experiment changes one thing relative to a baseline.
- Comparisons that change the batch keep the total token count fixed. Where relevant, they also keep the decay start fixed *in tokens*: with a batch ramp, a fraction of steps is not a fraction of tokens.

Decay branches.

- From a checkpoint of a constant-LR run at step S we run a short LR decay and record the final eval loss $R(S)$. This is the cooldown protocol of Hägele et al. (2024) and Hu et al. (2024); Merrill et al. (2025) use the same branching idea to measure the critical batch size.
- $R(S)$ removes the "noise floor", the part of the loss that any decay would remove immediately. What remains is real progress. This is the river–valley picture of WSD training (Wen et al., 2024): the constant-LR phase makes progress along the "river" while bouncing on the "hill" sides, and the decay removes the hill component.
- Comparing $R(S)$ curves between optimizers shows which one is genuinely ahead at each point.

Correlation checks. To ask whether a metric predicts final loss, we compute Spearman rank correlations:

- across all runs at one scale, mixing optimizers;
- within a single optimizer family.

4 Metric definitions

Notation.

- g: the gradient after clipping.
- $v = \Delta W$: the update the optimizer actually applied, including weight decay.
- W: the weights.
- $a = t - 1$ and $b = t$: two consecutive logged steps.

Each block type, or the whole model ("global"), is treated as one flattened vector, so group norms and cosines are exact.

Oscillation (`grad_cos_prev`; **Hu et al., 2024**)
$\cos(g_b, g_a) = \dfrac{g_b \cdot g_a}{\|g_b\| \, \|g_a\|}$. Negative values mean consecutive gradients point in opposite directions, i.e. the iterate bounces across a valley.

Update persistence (`upd_cos_prev`)
$\cos(v_b, v_a)$: how much each update repeats the previous one (momentum smoothing).

Preconditioning / alignment (`upd_grad_align`)
$\cos(v, -g)$. A value of 1 means the update is the plain gradient direction (SGD). Small values mean the optimizer rotates the step far from the raw gradient.

Relative update size (`upd_rel`)
$\|\Delta W\|_F / \|W\|_F$.

Overshoot slope (`D1`; **Hu et al., 2024, §4.4**)

$D_1 = \dfrac{g_b \cdot v_a}{\|v_a\|}$: the slope of the loss along the last step, measured after taking it. Positive values mean the step went past the minimum along its own direction. D_1 is not biased by batch noise, because the noise in g_b is independent of v_a.

Normalized overshoot (ov_n; **introduced in this work**)

$\mathrm{ov}_n = \dfrac{D_1/\|g\|}{\cos(v,-g)}$: overshoot relative to how much of the step was spent along the gradient. Raw $D_1/\|g\|$ mixes overshoot with alignment, so we use this normalized form.

Directional sharpness (`dir_sharpness`; D_2 **in Hu et al., 2024**)

$\dfrac{(g_b - g_a)\cdot v_a}{\|v_a\|^2} \approx \dfrac{v^\top H v}{\|v\|^2}$. This estimate is *biased*: batch noise adds a positive term that scales as $1/\mathrm{LR}$ (Section 6.6). The curvature $K = |D_2|/(1+D_1^2)^{3/2}$ (Hu et al., 2024) has the same bias.

Weight stable rank (`w_srank`; **Rudelson and Vershynin, 2007**)

$\|W\|_F^2/\|W\|_2^2$: the effective number of directions a weight matrix uses, averaged over the matrices in a block.

Update effective rank (`upd_erank_frac`; **Roy and Vetterli, 2007, in the form of Huang et al., 2026, §3.2**)

$\mathrm{erank}(A) = \exp\big(-\sum_i p_i \log p_i\big)$, with $p_i = \sigma_i^2/\sum_j \sigma_j^2$. It is reported as $\mathrm{erank}/\min(m,n)$ for the one-step update $A = \Delta W$. A value of 1 means the update is spread evenly over all directions.

- `delta_erank_frac`: the same quantity for the change accumulated over the logging interval, $W_t - W_{t-50}$.

Per-neuron imbalance (`upd_row_cv`)

Standard deviation divided by mean of the per-row update norms. This is the quantity NorMuon is designed to reduce (Li et al., 2025).

Weight-decay timescale (**Wang and Aitchison, 2024**; **Kosson et al., 2024**)

$\tau = 1/(\eta\lambda)$ steps, for learning rate η and weight decay λ: roughly how long decoupled weight decay needs to bring weight norms to equilibrium.

Gradient noise scale ("$B_{\mathbf{simple}}$"; **McCandlish et al., 2018**)

Take two independent micro-batches of b tokens at the same weights. Then

$$\widehat{|G|^2} = g_1 \cdot g_2, \qquad \widehat{\mathrm{tr}\Sigma} = b\Big(\tfrac{\|g_1\|^2+\|g_2\|^2}{2} - g_1\cdot g_2\Big), \qquad B_{\mathrm{noise}} = \frac{\mathrm{tr}\Sigma}{|G|^2}\ \text{(tokens)}.$$

B_{noise} is the batch at which gradient noise equals the true-gradient signal, a classical proxy for the critical batch size.

- Single measurements are heavy-tailed, so $|G|^2$ and $\mathrm{tr}\Sigma$ are averaged over 0.1B-token (1B runs) or 0.2B-token (3B runs) windows before dividing.
- Merrill et al. (2025) show that this estimate underestimates the critical batch.

Real progress $R(S)$ **and noise floor (after Hägele et al., 2024; Wen et al., 2024)**

$R(S)$ is the final eval loss of a decay branch started from step S. The noise floor is $\mathrm{eval}(S) - R(S)$.

The logger was validated on problems with known answers (Appendix D).

5 Which optimizer? (proxy and main models)

Five optimizers were trained with baseline settings across both the proxy and main models (Figure 1, Tables 6 and 7). The decay gain is the eval loss at the start of the decay minus the final eval loss.

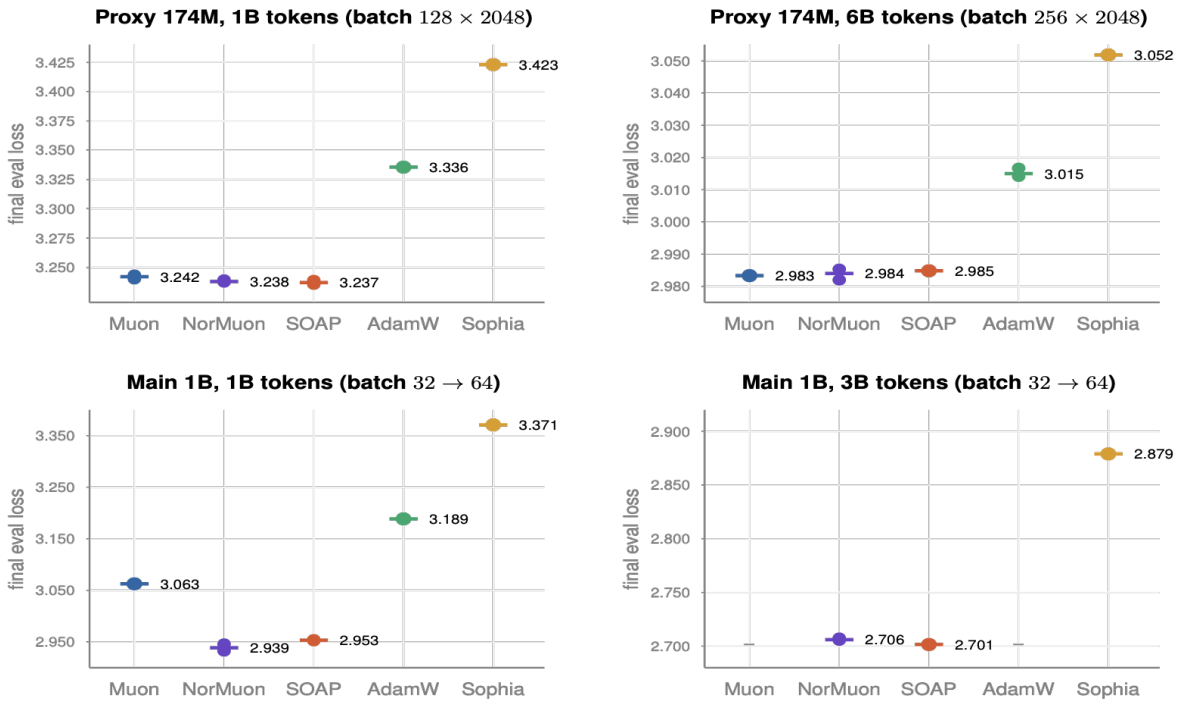


Figure 1: Final eval loss by optimizer on the proxy model (top) and the 1B-parameter main model (bottom). Dots are individual runs; the bar is their mean.

Table 6: Proxy-model final eval loss and decay gain. Two values are repeat runs. Decay gain is the eval loss at the start of the decay minus the final eval loss (reported for the first run where repeats exist).

Optimizer	Proxy-1B final	Proxy-1B gain	Proxy-6B final	Proxy-6B gain
NorMuon	3.2393 / 3.2372	0.164	2.9820 / 2.9853	0.183
Muon	3.2403 / 3.2436	0.164	2.9833	0.179
SOAP	3.2355 / 3.2389	0.157	2.9849	0.172
AdamW	3.3356	0.137	3.0142 / 3.0167	0.108
Sophia	3.4229	0.111	3.0519	0.087

- **NorMuon, Muon and SOAP are tied** within seed noise at both budgets. Wen et al. (2025) report the same pattern: matrix-based optimizers beat well-tuned AdamW by a margin that shrinks with scale, and differ little among themselves at this size. Essential AI et al. (2025) likewise report Muon ahead of AdamW in pretraining.
- **Caveat:** the proxy-1B AdamW and Sophia runs in Table 6 have norm-gain WD off, unlike the other three optimizers (Table 24). That accounts for at most ≈0.01 of their gap; norm WD's effect is measured directly in the next item.

Table 7: Main-model (Llama-3.2-1B) final eval loss and decay gain. Two values are repeat runs. Decay gain is the eval loss at the start of the decay minus the final eval loss (reported for the first run where repeats exist).

Optimizer	Main-1B final	Main-1B gain	Main-3B final	Main-3B gain
NorMuon	2.9324 / 2.9454	0.164	2.7075 / 2.7053	0.208
SOAP	2.9545 / 2.9522	0.150	2.7014	0.179
AdamW	3.1886	0.111	–	–
Sophia	3.3710	0.103	2.8790	0.098
Muon	3.0630	0.145	–	–

- **Weight decay on the norm gains** helps by about 0.01 at proxy-1B. Without it, Muon reaches 3.2494 and SOAP 3.2478. At proxy-6B the gap shrinks to ≈ 0.004.
- **On the main model** at 1B tokens with 10% decay:
  - The final eval losses are NorMuon 2.939 and SOAP 2.953 (means of two identical-flag runs each), while AdamW reaches 3.189 (weakly tuned; Table 23).
  - NorMuon leads SOAP by 0.015, and SOAP gains less in the decay (0.150 vs. 0.164).
  - At 3B tokens (single seeds, from `experiments.md`) SOAP leads, 2.7014 vs. 2.7075 (-0.006). It is ahead by 0.035 at the decay start, and NorMuon recovers most of the gap in the decay.
  - SOAP costs 20–40% more time per step on this model.

6 What the dynamics metrics show

6.1 Oscillation

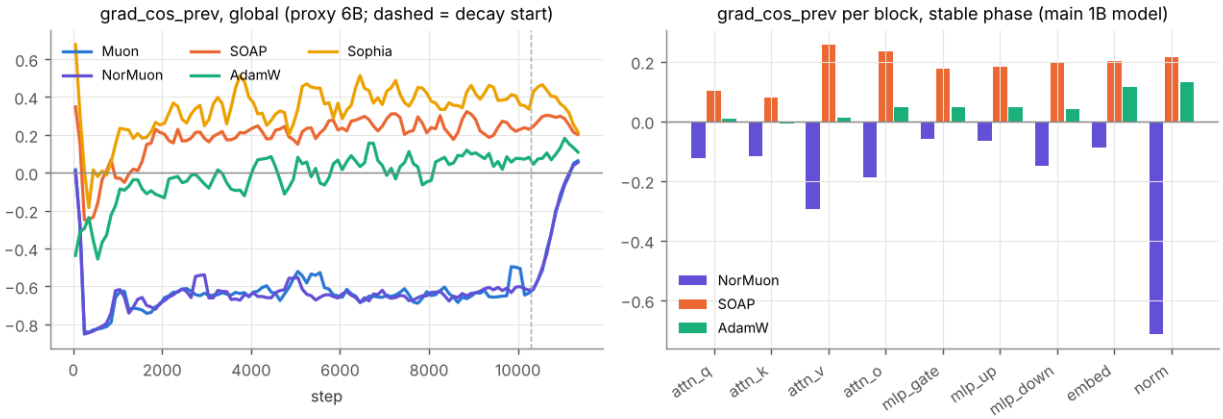


Figure 2: Left: global consecutive-gradient cosine over training (proxy-6B). Right: stable-phase value per block on the main model.

- **Muon-family optimizers oscillate; SOAP, AdamW and Sophia do not** (Figure 2, Table 8).
  - Under NorMuon every block has a negative cosine.

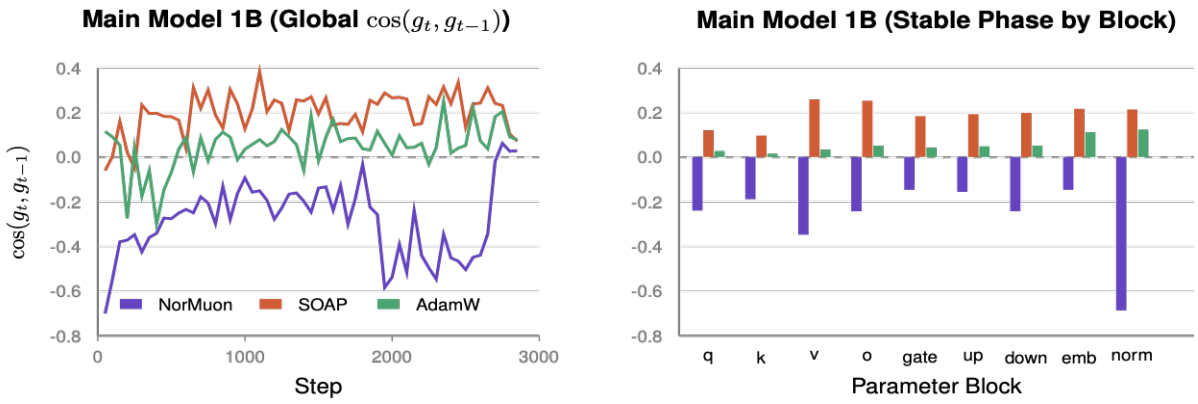


Figure 3: Main-model (1B) consecutive-gradient cosine: global curves over training (left) and stable-phase values per block (right).

Table 8: Stable-phase consecutive-gradient cosine (global).

	Muon	NorMuon	SOAP	AdamW	Sophia
Proxy-6B	-0.64	-0.65	$+0.24$	$+0.05$	$+0.40$
Main model	–	-0.19	$+0.21$	$+0.07$	–

  - The norm gains are the most negative (-0.78 proxy; -0.71 to -0.84 main), even though AdamW trains them inside a Muon run. In a SOAP or AdamW run the same norm gains are positive.

- **The oscillation disappears during the LR decay** (proxy-6B: $-0.6 \to +0.05$). It is a step-size effect: steps sized at the edge of stability (Cohen et al., 2021, 2022) bounce between the valley walls. This is the "hill" motion of the river–valley picture (Wen et al., 2024); Shen et al. (2026) analyse Muon in the same framework.
- **It is not caused by batch noise.** When the main-model batch doubles from 32 to 64, NorMuon's cosine goes from -0.21 to -0.47: less noise exposes more of the deterministic bounce.
- **Relation to final loss.**
  - Within the 13 Muon/NorMuon proxy runs, less oscillation goes with slightly lower loss ($\rho = -0.58$, $p = 0.04$). This is driven by two variants: removing norm WD (more oscillation, worse loss) and orthogonalizing Q/K/V per head (less oscillation, slightly better).
  - Across the 10 main-model NorMuon runs there is no relation ($\rho = 0.01$).

6.2 Preconditioning: how far the update departs from the gradient

- Every optimizer moves far from the raw gradient: alignment is 0.05–0.21 (Figure 4, Table 9).
- **SOAP and Sophia rotate the step the most**, and their updates are the most persistent from one step to the next.
- **Update rank:**

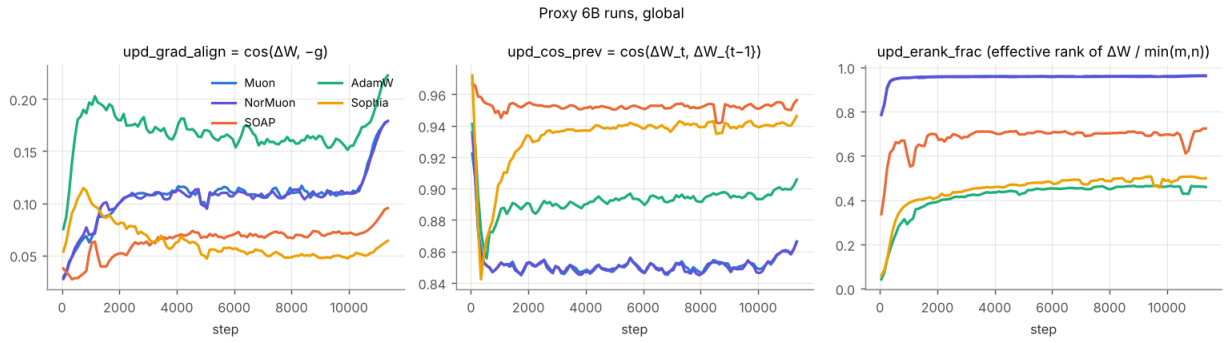


Figure 4: Alignment of the update with the negative gradient, update persistence, and update effective rank (proxy-6B, global).

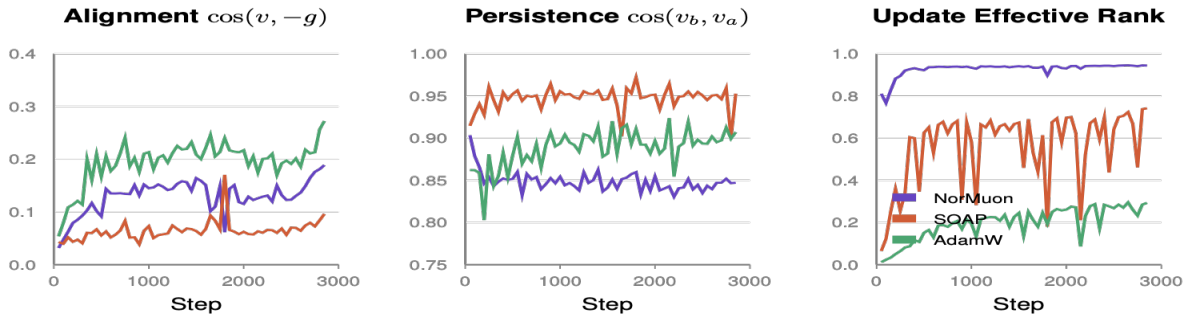


Figure 5: Alignment $\cos(v, -g)$, persistence $\cos(v_b, v_a)$, and update effective rank fraction on the main model (1B, global).

Table 9: Stable-phase preconditioning statistics: Proxy-6B and Main-1B.

Statistic	Muon	NorMuon	SOAP	AdamW	Sophia
Proxy model (6B tokens)					
$\cos(v, -g)$	0.111	0.108	0.070	0.162	0.053
$\cos(v_b, v_a)$	0.850	0.850	0.950	0.890	0.940
Update effective rank (fraction)	0.960	0.958	0.708	0.453	0.484
Main model (1B tokens)					
$\cos(v, -g)$	–	0.136	0.066	0.206	–
$\cos(v_b, v_a)$	–	0.846	0.950	0.895	–
Update effective rank (fraction)	–	0.938	0.588	0.224	–

  - Muon's orthogonalization makes each update almost full rank (0.96), as designed (Jordan et al., 2024).
  - SOAP is intermediate (0.71).
  - AdamW and Sophia concentrate their updates in fewer directions (0.45–0.48).
  - On the main model the spread is wider: NorMuon 0.94, SOAP 0.59, AdamW 0.22.
- Alignment rises sharply during the decay for every optimizer.

6.3 Overshoot

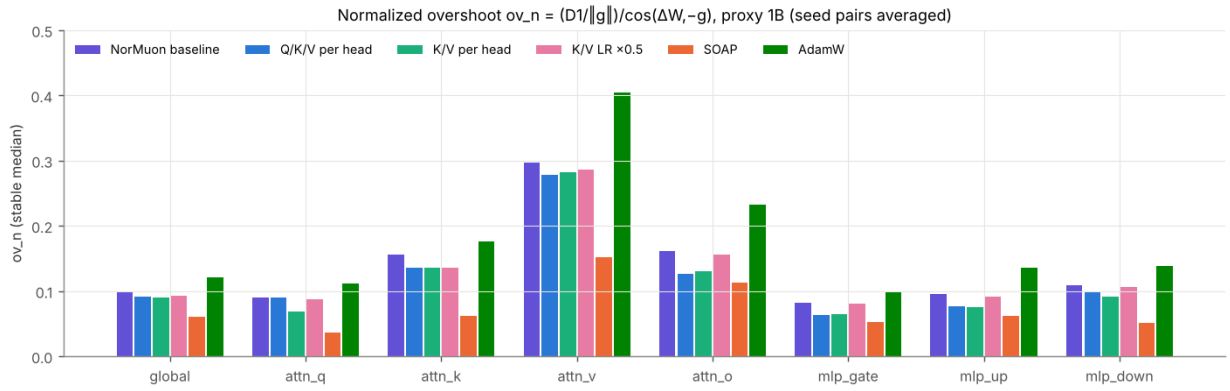


Figure 6: Normalized overshoot ov_n per block on proxy-1B (seed pairs averaged).

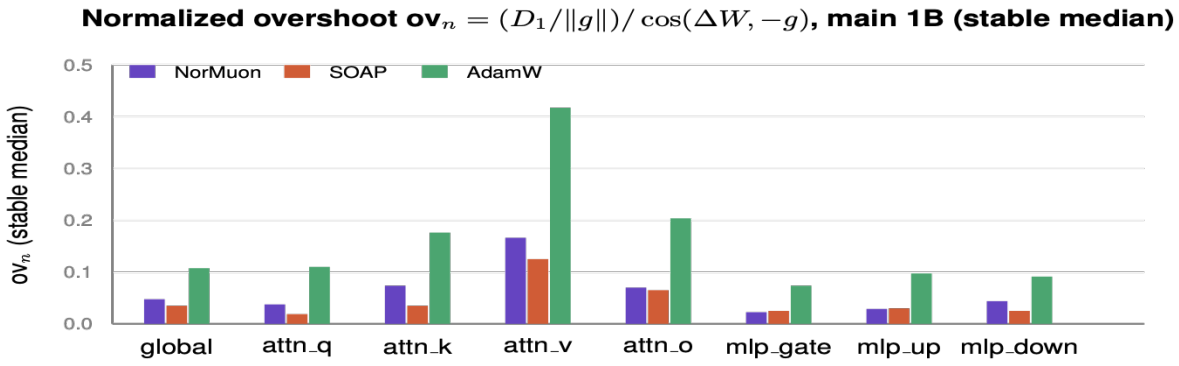


Figure 7: Normalized overshoot ov_n per block on the main model (1B, stable phase).

- D_1 **is positive at 59 of 59 stable-phase measurements for every optimizer.** Every step goes past the minimum along its own direction, i.e. all optimizers train at the edge of stability (Cohen et al., 2021 for gradient descent; Cohen et al., 2022 for adaptive methods).
- **By block** (Figure 6 and Figure 7, Table 10):
  - `attn_v` **overshoots most** under every optimizer (NorMuon 0.18/0.30, AdamW 0.40/0.40), followed by `attn_k` and `attn_o`.

Table 10: Normalized overshoot ov_n per block: Proxy-1B and Main-1B.

Block	Muon	NorMuon	SOAP	AdamW	Sophia
Proxy model (1B tokens)					
global	**0.101**	**0.100**	**0.060**	**0.121**	**0.138**
`attn_q`	0.086	0.092	0.036	0.122	0.100
`attn_k`	0.150	0.149	0.061	0.199	0.190
`attn_v`	**0.286**	**0.301**	**0.147**	**0.404**	**0.503**
`attn_o`	0.158	0.164	0.108	0.235	0.292
`mlp_gate`	0.084	0.086	0.055	0.108	0.104
`mlp_up`	0.098	0.099	0.064	0.146	0.148
`mlp_down`	0.111	0.112	0.055	0.145	0.176
`embed`	0.029	0.029	0.027	0.030	0.010
`norm`	0.491	0.513	0.425	0.275	0.275
Main model (1B tokens)					
global	–	**0.048**	**0.036**	**0.108**	–
`attn_q`	–	0.041	0.018	0.110	–
`attn_k`	–	0.077	0.033	0.178	–
`attn_v`	–	**0.178**	**0.129**	**0.405**	–
`attn_o`	–	0.072	0.064	0.199	–
`mlp_gate`	–	0.025	0.024	0.076	–
`mlp_up`	–	0.029	0.030	0.099	–
`mlp_down`	–	0.044	0.025	0.095	–
`embed`	–	0.031	0.031	0.032	–
`norm`	–	0.354	0.311	–	–

  - SOAP has the lowest overshoot in every block.

- Orthogonalizing Q/K/V per head lowered NorMuon's overshoot (global $0.100 \to 0.091$; attn_o $0.16 \to 0.13$) **without changing the loss** (Section 7).
- **Overshoot does not predict final loss across optimizers.** Its correlation with loss is $\rho = +0.62$ at proxy-1B but $\rho = -0.55$ at proxy-6B.

6.4 Rank, and how rank relates to final loss

Table 11: Stable-phase weight stable rank.

	Muon / NorMuon	SOAP	AdamW	Sophia
Proxy-1B, hidden matrices	115	113	49	25
Proxy-6B, hidden matrices	111	111	59	46
Main model, hidden matrices	333	292	164	–
Main model, embedding	3.6	**42**	4.3	–

- **Across optimizers, higher rank goes with lower loss** (Figures 8–9, Tables 11–12). The correlation comes from the optimizer clusters: Sophia $<$ AdamW $<$ SOAP $\approx$ Muon in rank, with loss in the reverse order.
- **Within one optimizer, rank barely moves and does not track loss.**

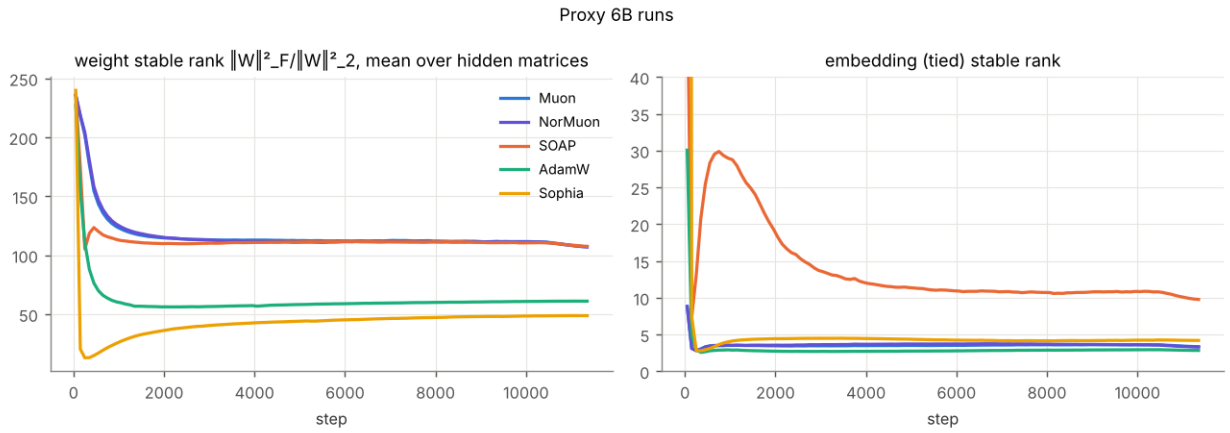


Figure 8: Weight stable rank of hidden matrices (left) and of the tied embedding (right), proxy-6B.

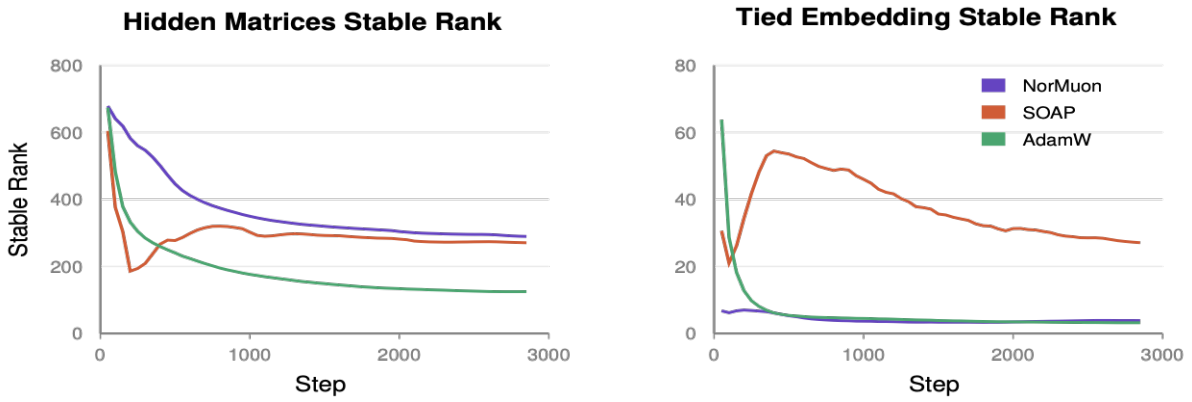


Figure 9: Weight stable rank of hidden matrices (left) and of the tied embedding (right) on the main model (1B).

Table 12: Spearman correlation between stable-phase rank and final eval loss.

Group (runs)	Weight stable rank	Accumulated-update rank	One-step update rank
Proxy-1B, all optimizers (22)	-0.44 ($p = 0.04$)	-0.39	-0.20
Proxy-6B, all optimizers (12)	$\mathbf{-0.95}$ ($p < 0.001$)	$\mathbf{-0.93}$	-0.73
Main model, all 1B-token runs (14)	-0.37	$\mathbf{-0.68}$ ($p = 0.007$)	-0.69
Within Muon/NorMuon, proxy-1B (13)	-0.11 (n.s.)	-0.01 (n.s.)	$+\mathbf{0.65}$ (wrong sign)
Within NorMuon, main model (10)	$+0.35$ (n.s.)	-0.29 (n.s.)	-0.34 (n.s.)

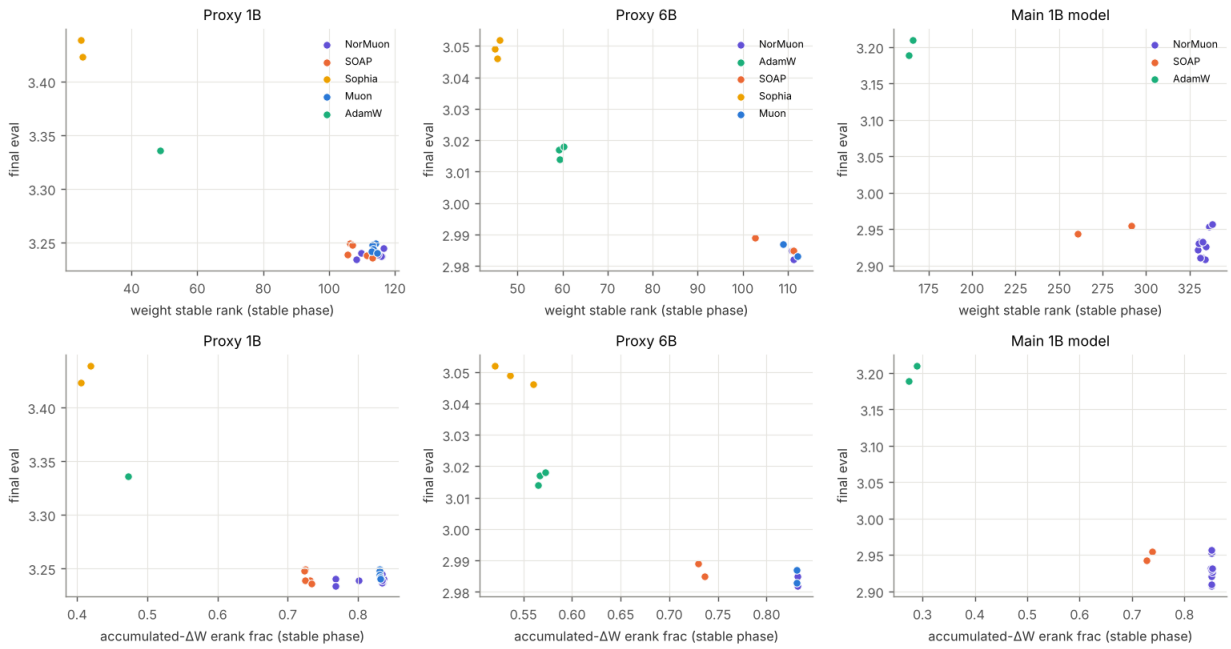


Figure 10: Stable-phase weight stable rank (top) and accumulated-update effective rank (bottom) against final eval loss, coloured by optimizer.

  - The 10 main-model NorMuon runs span update rank 0.935–0.939 while their final loss spans 2.908–2.957.
  - Where rank does move within the family, it moves in the opposite direction: per-head Q/K/V lowers update rank from 0.96 to 0.82, yet gives the (insignificantly) best loss.

- **SOAP's embedding has ten times the stable rank of the others.** This did not translate into better loss, and giving NorMuon a SOAP-trained embedding did not help either (Section 7).
- **Conclusion:** rank identifies the optimizer family; it is not a target to tune within a family.

6.5 Norm layers: update speed is set by LR × WD

Table 13: Norm-layer dynamics: relative update size $\|\Delta W\|/\|W\|$ and weight norm $\|W\|_F$ shrinkage (start $\rightarrow$ end).

	Proxy Model (6B tokens)		Main Model (1B tokens)	
Optimizer	Stable $\|\Delta W\|/\|W\|$	$\|W\|_F$ shrinkage	Stable $\|\Delta W\|/\|W\|$	$\|W\|_F$ shrinkage
Muon	1.1×10^{-3}	$138.3 \rightarrow 56.9$	–	–
NorMuon	1.1×10^{-3}	$138.3 \rightarrow 56.7$	0.9×10^{-3}	$258.1 \rightarrow 139.2$
SOAP	1.8×10^{-3}	$138.4 \rightarrow 36.3$	0.9×10^{-3}	$257.9 \rightarrow 85.1$
AdamW	0.3×10^{-3}	$138.5 \rightarrow 70.4$	0.1×10^{-3}	$259.8 \rightarrow 241.1$
Sophia	0.1×10^{-3}	$138.6 \rightarrow 80.0$	–	–

- SOAP appears to update the norm gains fastest in *relative* terms ($\approx 1.8 \times 10^{-3}$ per step on proxy vs. Muon's 1.1×10^{-3}; Figure 11 and Figure 12, Table 13). On the main model, NorMuon and SOAP tie

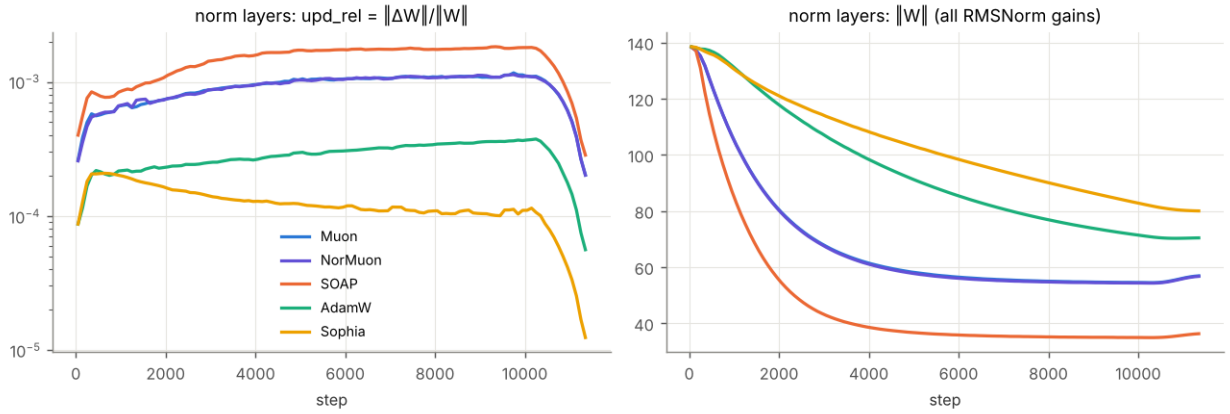


Figure 11: Norm-gain relative update size (left) and norm-gain weight norm (right), proxy-6B.

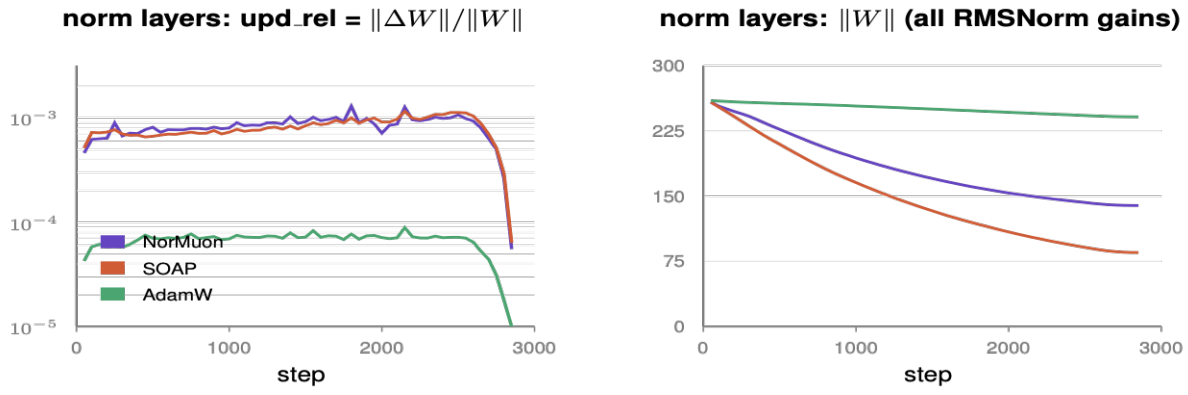


Figure 12: Norm-gain relative update size (left) and norm-gain weight norm (right) on the main model (1B).

at $\approx 0.9 \times 10^{-3}$ because both use norm-gain LR 3×10^{-3}.

- This comes from the denominator: weight decay shrinks the gains the most when LR is high. On the main model, norm weight shrinks from 258 to 85 (SOAP) and 139 (NorMuon), while AdamW's weight decay is inactive (260 $\rightarrow$ 241). On the proxy, norms shrank to 36 (SOAP), 57 (Muon), 70 (AdamW), and 80 (Sophia).
- Within one optimizer, the absolute norm-gain update is the same with or without norm WD (SOAP 0.050 vs. 0.052; Muon 0.059 vs. 0.061).
- Lowering the norm-gain LR in a NorMuon run from 0.003 to 0.001 made the loss worse (+0.007).

6.6 The rise in sharpness during the decay is a measurement artifact

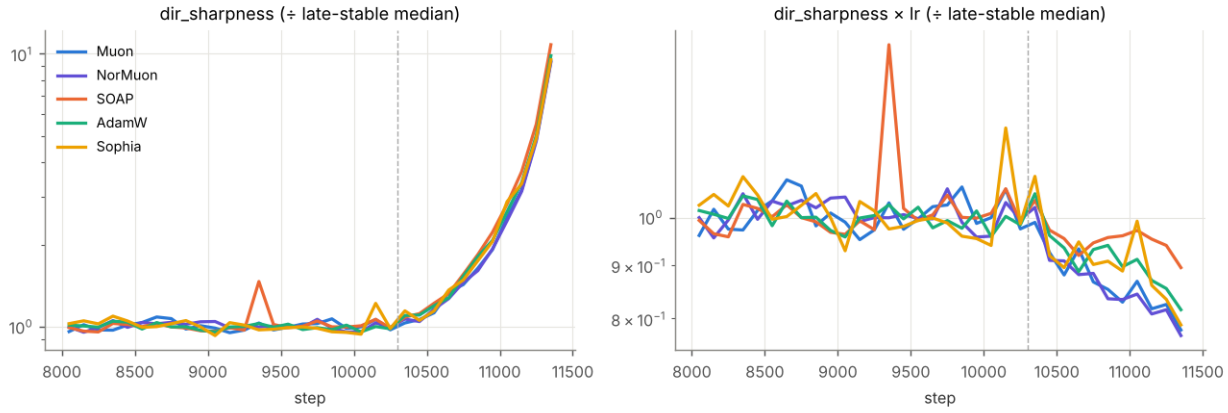


Figure 13: Directional sharpness (left) and sharpness $\times$ LR (right) through the decay, each normalized to its late-stable median. Dashed line: decay start.

- The two-batch directional sharpness rises about 10$\times$ during the decay for every optimizer (Figure 13, left).
- **Sharpness $\times$ LR stays roughly flat** (Figure 13, right), and even drifts down 20–25%:
  - AdamW 1.6 $\rightarrow 1.2 \times 10^{-5}$;
  - Muon 4.2 $\rightarrow 3.0 \times 10^{-5}$;
  - SOAP 4.9 $\rightarrow 3.7 \times 10^{-6}$.
- The cause is a bias in the estimator as defined by Hu et al. (2024). The step from batch a contains part of batch a's noise, which inflates the estimate by a term that scales like $1/\text{LR}$. The true curvature along the step does not grow.
- The derivation and a numerical check are ours (Appendix D). Curvature questions should instead use a probe metric that evaluates both gradients on the same fixed batch.

6.7 Weight-decay timescale

- On the main model, AdamW's $\tau \approx 33$k steps, far longer than the 2,860-step run. **Its weight decay is therefore effectively inactive** (Figure 14, Table 14).

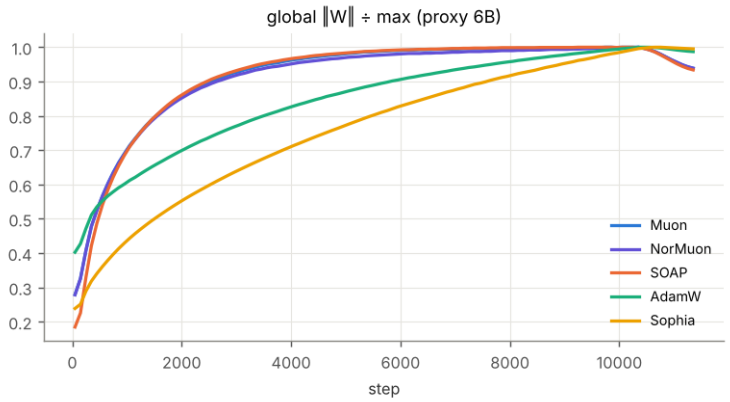


Figure 14: Global weight norm divided by its maximum, proxy-6B.

Table 14: Weight-decay timescale and equilibrium, proxy-6B.

Optimizer	$\tau = 1/(\eta\lambda)$	Weight norm reaches equilibrium at
NorMuon / Muon	$\approx$1k steps	$\sim$4k steps
SOAP	2.5k	$\sim$6k
AdamW	10k	not reached by 11.4k
Sophia	–	not reached by 11.4k

- This is one reason AdamW lags and benefits less from a longer decay.
- Wang and Aitchison (2024) recommend choosing weight decay so that τ is a fixed fraction of the run.

7 Experiment: per-block changes to NorMuon (proxy-1B)

The baseline is the NorMuon proxy baseline, with a two-seed mean of 3.2383. Each variant in Table 15 changes one thing. Using different optimizers or settings for different parameter groups is standard Muon practice (Jordan et al., 2024; Liu et al., 2025), where only hidden matrices use Muon.

Table 15: Per-block variants of the NorMuon baseline (proxy-1B).

Variant (change from baseline)	Final eval	Δ	Main dynamics change
Orthogonalize Q, K and V *per attention head* instead of as whole matrices (2 seeds)	3.2339 / 3.240	-0.001	overshoot 0.100 $\rightarrow$ 0.091; alignment 0.119 $\rightarrow$ 0.162; oscillation -0.47 $\rightarrow$ -0.36; update rank 0.956 $\rightarrow$ 0.818
Orthogonalize only K and V per head	3.2390	$+0.001$	–
Halve the LR of the K and V matrices	3.2375	-0.001	–
Train the embedding with SOAP instead of AdamW	3.2400	$+0.002$	embedding stable rank 2.7 $\rightarrow$ 9.9
Norm-gain LR 0.001 instead of 0.003	3.2453	$+0.007$	–

- **No variant beats seed noise.** Per-head Q/K/V looked like -0.0044 on its first seed; the second seed erased it.
- **The same five variants on the main model at 3B tokens** all land within ± 0.004 of the baseline

(2.7075). The ordering matches the proxy (per-head Q/K/V best, -0.004; SOAP embedding worst, $+0.004$), but every gap is noise-sized.

- Most of each variant's lead disappeared after the decay. Per-block changes mostly moved the noise floor, not real progress.
- The variants clearly changed the dynamics metrics but not the loss. Within a family, those metrics are not reliable guides.

8 Experiment: switch from NorMuon to AdamW mid-run? (proxy-6B)

Method.

- The NorMuon and AdamW proxy-6B baselines were checkpointed at steps 3000, 5500 and 8000.
- From each checkpoint we ran a decay branch with the same decay as the full run.
- The full run provides the fourth point, with its decay starting at step 10,300.

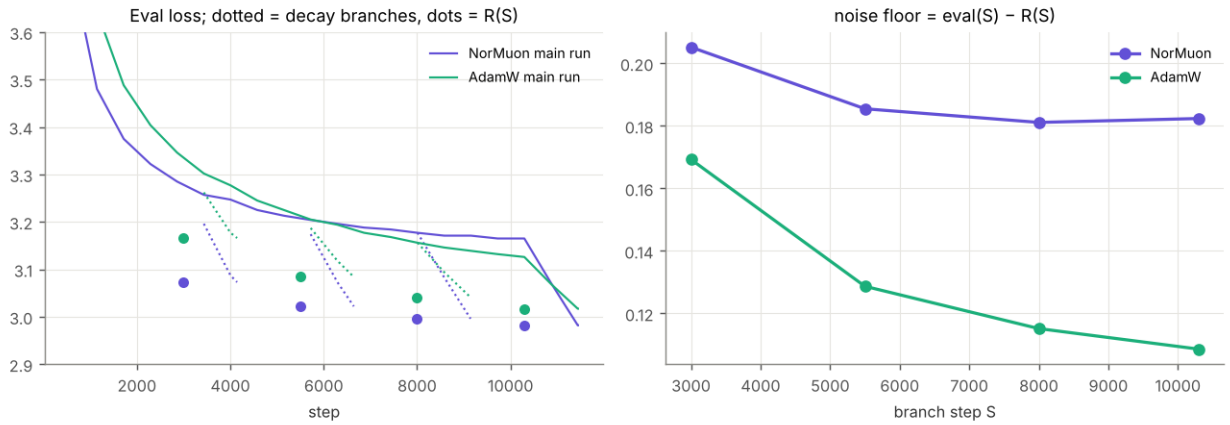


Figure 15: Left: eval loss of the full runs (solid), decay branches (dotted) and $R(S)$ (dots). Right: noise floor $\text{eval}(S) - R(S)$.

Table 16: Real progress $R(S)$ and noise floor at each branch point.

Step S	NorMuon $R(S)$	AdamW $R(S)$	NorMuon floor	AdamW floor
3,000	3.0726	3.1660	0.206	0.170
5,500	3.0216	3.0841	0.186	0.128
8,000	2.9959	3.0408	0.181	0.115
10,300	2.9820	3.0167	0.183	0.110

- **Fits** of $R(S) = a + bS^{-c}$ (S in thousands of steps; Figure 15, Table 16):
  - NorMuon: $a = 2.869$, $b = 0.345$, $c = 0.481$;
  - AdamW: $a = 2.770$, $b = 0.605$, $c = 0.385$.
- At matched R, AdamW improves only 0.59–0.77$\times$ as fast as NorMuon. **There is no point at which a switch would pay**, so the switch run was not needed.

- NorMuon carries a larger noise floor (≈0.18–0.21 vs. 0.11–0.17). This explains why it gains more from the decay, and it predicts that NorMuon benefits more from a longer decay (Section 9).

9 Experiment: decay length and shape (main model, 1B tokens)

Each optimizer's baseline (10% linear decay) was compared with a decay over 30% of steps (Figure 16, Table 17). Because the batch doubles at 50% of tokens, 30% of *steps* is 45% of *tokens*.

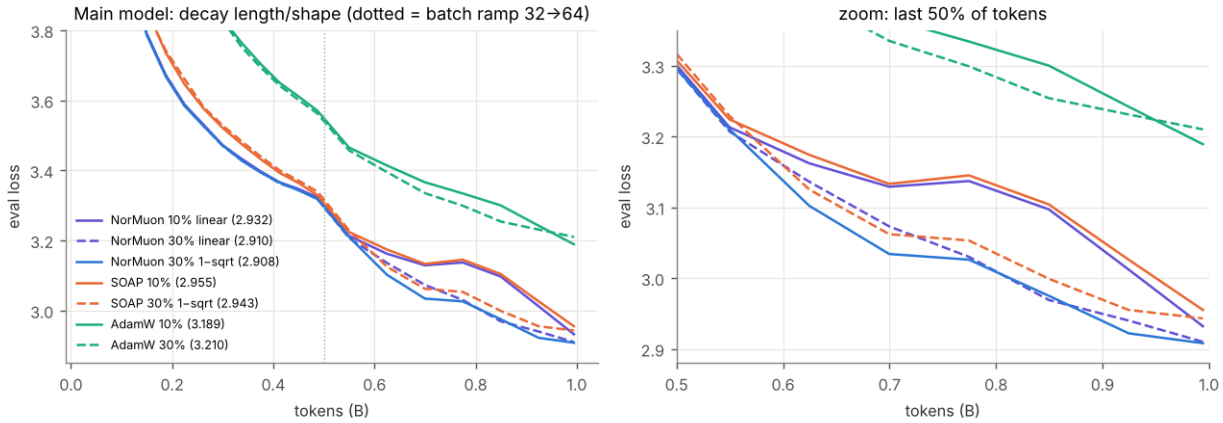


Figure 16: Eval loss against tokens for each decay variant (right: last 50% of tokens). Dotted line: batch increase from 32 to 64.

Table 17: Decay length and shape on the main model.

Optimizer	Decay	Final	Decay gain
NorMuon	10% linear (baseline; 2 identical runs)	2.932 / 2.945	0.165
NorMuon	30% linear	2.910	0.297
NorMuon	30% 1−sqrt (2 seeds)	2.908 / 2.910	0.30
SOAP	10% linear (baseline; 2 identical runs)	2.955 / 2.952	0.149
SOAP	30% 1−sqrt	2.943	0.286
AdamW	10% linear (baseline)	3.189	0.111
AdamW	30% linear	3.210	0.247

- **The longer decay helps NorMuon most and hurts AdamW.** Relative to the 10%-decay means:
  - NorMuon improves by 0.030, well beyond the 0.013 spread.
  - SOAP improves by 0.011, within the spread.
  - AdamW gets worse by 0.021.

  The 30% decay also shrinks seed-to-seed spread from 0.013 to 0.002–0.004.
  - This ordering matches the noise-floor ordering of Section 8: the larger the removable floor, the more a long decay helps.
  - AdamW loses because the longer decay shortens its constant-LR phase while its weight decay is inactive (Section 6.7).

- Linear decay and the 1−sqrt shape proposed by Hägele et al. (2024) are equivalent here (−0.002).
- Hägele et al. (2024) also found that longer cooldowns, up to ≈20% of training, help. Our 30%-of-steps result extends this to NorMuon.
- With a changing batch, the decay length should be specified in tokens, not steps.

10 Experiment: batch size and batch schedule (main model)

10.1 At 1B tokens

Setup. The baseline is NorMuon with the best decay from Section 9: 30% of steps, 1−sqrt shape. The batch goes from 32 to 64 at 50% of tokens, and the decay starts at 0.55B tokens. Each variant has two seeds (Figure 17, Table 18).

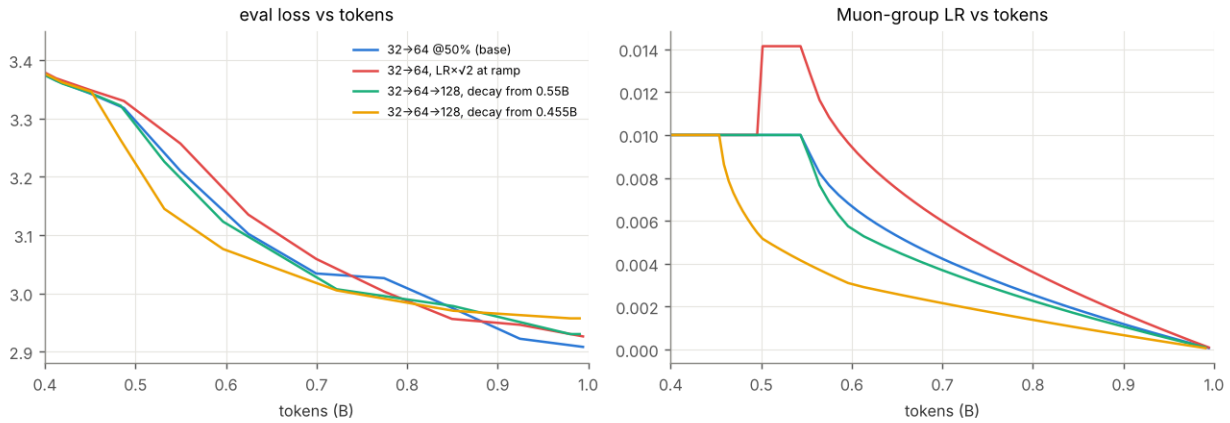


Figure 17: 1B-token batch-schedule variants: eval loss (left) and NorMuon learning rate (right) against tokens.

Table 18: 1B-token batch-schedule variants (two seeds each).

Variant (change from baseline)	Decay starts at	Final	Δ
Baseline	0.55B tokens	2.908 / 2.910	–
Raise NorMuon's LR by $\sqrt{2}$ (0.010 → 0.0141) when the batch doubles	0.55B	2.926 / 2.921	+**0.015**
Second doubling to 128 at 55% of tokens; decay start kept at 0.55B tokens, so the decay runs mostly at 128	0.55B	2.930 / 2.932	+**0.022**
Same 128 schedule, but decay over 30% of steps, so it starts before the batch increase	0.455B	2.957 / 2.953	+0.046

- **Doubling the batch acts like a partial LR decay.**
  - In the baseline schedules, the 32 → 64 step alone produces a one-time train-loss drop: NorMuon −0.068, SOAP −0.057, AdamW −0.031, which accounts for 41% / 38% / 28% of each optimizer's whole 10%-decay gain.
  - Halving the gradient noise lowers the noise floor, just as an LR cut does. This is the equivalence between batch increases and LR decay described by Smith et al. (2018) and refined by Meterez et al. (2025).

- **Compensating with higher LR hurts.**
  - Raising the LR by $\sqrt{2}$ when the batch doubles keeps the per-step noise level roughly constant. This follows the square-root scaling rule for adaptive optimizers (Malladi et al., 2022) and the LR–batch trade-off of Meterez et al. (2025).
  - Doing so throws away the free annealing and costs 0.015.
- **Running the decay at batch 128 costs 0.022** at this budget. Starting the decay before the batch increase is worse still.
- NorMuon's oscillation strengthens when the batch doubles ($-0.21 \rightarrow -0.47$), consistent with Section 6.1.

10.2 At 3B tokens: a second doubling

Setup.

- Baseline (A): NorMuon, batch $32 \rightarrow 64$ at 50% of tokens (1.5B).
- Decay: linear over the last 0.45B tokens (15%), starting at 2.55B tokens.
- All variants keep the same token budget and the same decay window in tokens. All results are single seeds.
- Several variants are *branches*: they resume from another run's checkpoint and change only what comes after (Figure 18, Tables 19–20).

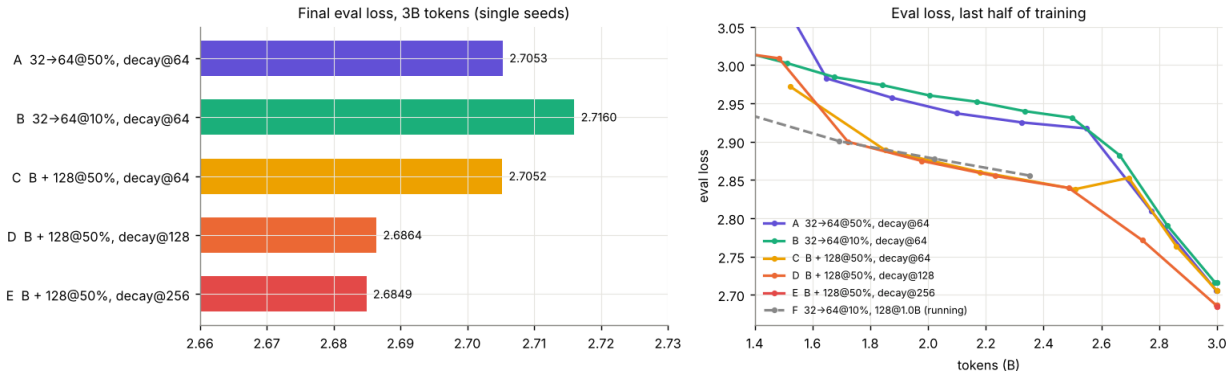


Figure 18: 3B-token batch-schedule variants: final eval loss (left) and eval loss over the last half of training (right).

- **Fewer, larger steps in the decay win at 3B tokens.** Decaying at 128 uses half the steps and is 0.019 better; at 256, with a quarter of the steps, it is just as good.
- **Cutting the batch at the decay start temporarily raises the loss.** In C, eval loss rises from 2.838 to 2.853 after the batch drops from 128 to 64 at 2.55B tokens, before the LR decay brings it down. This mirrors the drop seen when the batch increases.
- **The same change (decaying at 128) lost at 1B tokens.** Three explanations remain open:
  1. The useful batch grows as training progresses. The 1B-budget decay starts at 0.55B tokens (loss ≈ 3.2); the 3B-budget decay starts at 2.55B tokens (loss ≈ 2.9).

Table 19: 3B-token batch-schedule variants (batch in sequences of 8,192 tokens; single seeds).

	Variant	Decay batch (steps)	Final
A	Baseline: 32 → 64 at 1.5B tokens	64 (858)	2.7053
B	32 → 64 early, at 0.3B tokens	64 (857)	2.7160
C	B, then 128 from 1.5B to 2.55B tokens (branch of B at 1.5B)	back to 64 (857)	2.7052
D	B, then 128 from 1.5B tokens through the decay	128 (428)	**2.6864**
E	As C/D up to 2.55B, then decay at 256 (branch of C)	256 (214)	**2.6849**
F	B, then 128 from 1.0B tokens (branch of B at 1.0B)	128	running (2.856 at 2.35B tokens)
–	B with the decay at 32 (branch of B)	32	crashed

Table 20: Effect of each change in the 3B-token study.

Change	Comparison	Δ final
Increase to 64 early (0.3B instead of 1.5B tokens)	B − A	+0.011
Train at 128 instead of 64 between 1.5B and 2.55B tokens	C − B	−0.011
Decay at 128 instead of 64	D − C	**−0.019**
Decay at 256 instead of 128	E − D	−0.0015 (noise)
Best vs. baseline	E − A	**−0.020**

2. The decay schedules differ: 1−sqrt over 45% of tokens at 1B, linear over 15% at 3B.
3. The training history differs. At 3B the model trained 1B tokens at batch 128 before the decay; at 1B the switch to 128 coincides with the decay.

When should the first doubling (32 → 64) happen? A switch-point sweep answers this directly.

- **Method.** One batch-32 trunk run was checkpointed at several token counts, and each checkpoint was branched into a run that continues at 64. The LR schedule was defined in tokens, so every branch shares the trunk's LR.
- **1B budget.** Best switch at 0.5–0.67B tokens (2.945). Switching at 333M costs 0.012; fixed 64 costs 0.046; fixed 32 costs 0.038.
- **3B budget.** Best switch at 1.5B tokens (2.728), flat within 0.003 from 1B to 2B. A fixed-count rule would put the optimum at 0.5B, which is 0.009 worse.
- **Conclusion.** The best first doubling scales with the *budget* (about half of it), not with a fixed token count. Our default `32:0.5,64` follows this.

Does the useful batch depend on tokens seen?

- Theory and prior measurements say the critical batch grows as the loss falls (McCandlish et al., 2018; Merrill et al., 2025), which is why large runs increase the batch in stages (Grattafiori et al., 2024). Sato et al. (2025) derive a critical batch size specifically for Muon.
- For the first doubling, the switch-point sweep says the timing follows the budget fraction.
- For the second doubling, the evidence points the other way:
  - at 3B tokens, 128 from 1.5B tokens helps (−0.011 in the stable phase, −0.019 in the decay);

  - at 1B tokens, 128 in the decay hurts (+0.022).

- Both observations fit if the decay benefits from a larger batch only once enough training has accumulated. They also fit the decay-schedule confound listed above. Variant F (128 from 1.0B tokens) will help separate these explanations.

10.3 Gradient noise scale

The noise scale was measured in all runs of this section; the proxy runs did not log it.

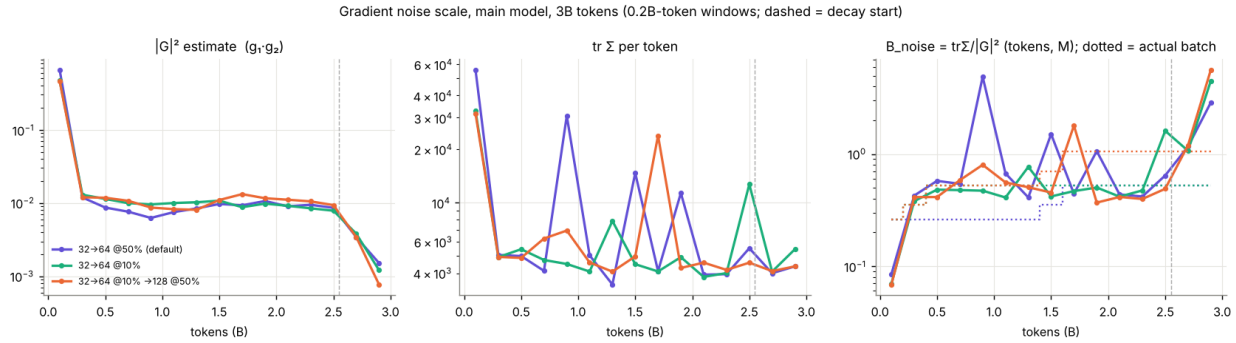


Figure 19: Noise-scale decomposition for three 3B-token runs: $|G|^2$ (left), $\mathrm{tr}\,\Sigma$ per token (middle), and B_{noise} with the actual batch dotted (right). Dashed line: decay start.

Table 21: Stable-phase gradient noise scale (median of 0.2B-token windows).

Run	Stable-phase B_{noise}	Actual batch in that phase
A, baseline (3B)	542k tokens	262k → 524k
B, early 64 (3B)	471k	524k
D, 128 from 1.5B (3B)	453k	524k → 1,049k
C and F, windows at batch 128 only	358k / 351k	1,049k
1B-budget runs (0.25–0.45B tokens)	320k–509k	262k

- **The noise per token, $\mathrm{tr}\,\Sigma$, is constant** (Figure 19, Table 21).
  - It stays at ≈ 4.0–5.0×10^3 from 0.2B tokens to the end, in every run, at every batch and in every phase.
  - Isolated windows at 10^4–10^6 are heavy-tailed single measurements.
  - All movement in the noise scale therefore comes from the signal term $|G|^2$.
- **In the 3B runs the noise scale is flat for most of training.**
  - It rises about $10\times$ in the first 0.2B tokens.
  - It then stays at 0.4–0.6M tokens, growing only $\approx 1.3\times$ between 0.5B and 2.5B tokens.
  - Read literally, a 1M-token batch is twice too large throughout the constant-LR phase, yet the runs using it (D, E) are the best.
- **A larger batch lowers the measured noise scale.** At matched tokens, $|G|^2 \approx 0.008$–0.010 at batch 32/64 and ≈ 0.012–0.014 at 128.

  - Our interpretation: with less noise, NorMuon's fixed-size steps follow the deterministic wall-to-wall bounce more cleanly (Section 6.1). The iterate sits higher on the valley walls, so the gradient at the iterate is larger.
  - Raising the LR has the same effect, tentatively, since this rests on three measurements per run. Right after the batch doubles, the $\sqrt{2}$-LR runs show $|G|^2 = 0.017$–0.024 vs. 0.0096 for the baseline.
  - Under NorMuon at constant LR, $|G|^2$ thus mostly measures this bounce, not how much progress is left. The resulting noise scale depends on the LR and on the batch itself.
- **During the decay $|G|^2$ collapses 5–10$\times$** as the bounce dies out, and the noise scale jumps to 1–10M tokens, above every batch tried.
  - Read literally, this recommends a larger decay batch. That matched the 3B result but not the 1B result.
  - In the last 1B-budget windows $|G|^2 \approx 0$ within noise, so the estimate is undefined there.
- **By block** (Figure 20):
  - attn_v has the lowest noise scale among the weight matrices (≈0.3–0.4M tokens), and it is also the block that overshoots most.
  - attn_k, mlp_gate and mlp_up are highest (≈0.9–1.2M).
  - The norm gains are lowest of all (≈50–70k).

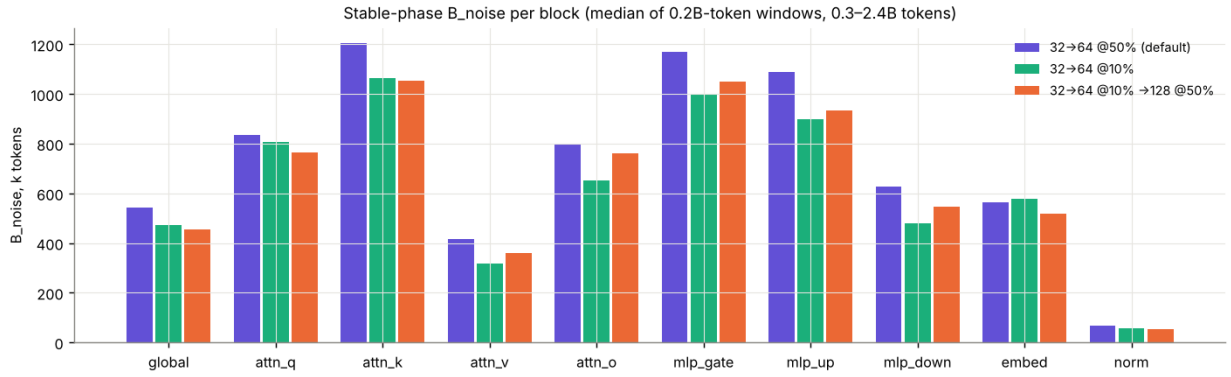


Figure 20: Stable-phase noise scale per block for three 3B-token runs.

Verdict on batch-doubling timing. In the previous section, we determined the timing for doubling the batch size by sweeping over different switch points and observing the final loss. However, the result from that procedure only partly agrees with the noise-scale estimate.

- **At 1B tokens the noise scale times the first doubling correctly.** Pooled over three runs, it grows from ≈0.12M tokens early to ≈0.48M by 0.45B tokens. It reaches the batch-64 size (524k tokens) at 0.42–0.5 of the budget, inside the switch-point sweep's flat optimum (0.33–0.67).
- **At 3B tokens it does not.** It reaches 0.4–0.6M tokens by ≈0.3B tokens and stays there, which would call for an early switch to 64. That switch costs 0.011 (variant B), and the best 3B schedule trains at 2–3$\times$ the measured noise scale.

- This agrees with Merrill et al. (2025), who found that the noise-scale estimate (McCandlish et al., 2018) underestimates the useful batch.
- Here the estimate is also batch- and LR-dependent. To our knowledge this has not been reported for Muon-family optimizers.
- We recommend using the noise scale as a diagnostic: phase changes and which blocks are noisiest. For batch timing beyond the first doubling at small budgets, use direct comparisons such as switch-point branches.

11 Which measurements helped which decisions

Table 22: Summary of which measurements informed each decision.

Decision	Measurement used	Verdict
Optimizer family	Final loss; rank and alignment as fingerprints	Rank separates families and tracks loss, but only across families.
Per-block settings	Overshoot, oscillation, per-neuron imbalance	Metrics moved but loss did not. **Not predictive within a family.**
Optimizer switch	$R(S)$ from decay branches	Decisive: no switch. Branches beat any single-run metric.
Decay length	Noise floor $\text{eval}(S) - R(S)$	Predictive: a larger floor means a larger gain from a longer decay.
Batch schedule; LR at batch increase	Loss drop at the increase, oscillation, noise scale	The first doubling should come at about half the budget (switch-point branches). At 1B tokens the increase is a partial anneal, and raising LR hurts; at 3B tokens a larger batch late and in the decay helps. The noise scale times the first doubling at 1B tokens but **not** at 3B tokens or in the decay.
Curvature during decay	Two-batch sharpness and curvature	Biased ($\propto 1/\text{LR}$); use a probe-batch metric.
Weight decay	$\tau = 1/(\eta\lambda)$, weight norm	Explains why AdamW's WD is inactive on the main model.

12 Limitations

- **Severe under-training relative to scaling laws.** The model sizes and token budgets in this study are far below what compute-optimal scaling laws recommend. Compute-optimal training (Chinchilla; Hoffmann et al., 2022) dictates approximately 20 tokens per parameter ($\geq$ 20B tokens for a 1B model), while practical pretraining regimes push overtraining significantly further (e.g., Llama-3.2-1B trained on 9T tokens, $\sim 9{,}000\times$; Grattafiori et al., 2024). In contrast, our proxy model (174M) was trained on only 1B–6B tokens ($5.7\times$–$34.5\times$), and our main model (1B) was trained on only 1B–3B tokens ($1\times$–$3\times$). Because both models were trained in this heavily token-starved regime, they remained entirely within early transient optimization dynamics. **Consequently, the results of hyperparameter tuning and optimizer comparisons may be heavily biased**: optimal learning rates, batch-schedule advantages, the large gain from extending the LR decay, and the apparent gaps between optimizer families reflect early-phase training behavior and may not transfer to compute-optimal or over-trained scaling frontiers.

- **Inactive weight decay in the AdamW baseline.** Decoupled weight decay on the main model had an equilibrium timescale of $\tau = 1/(\eta\lambda) \approx 33$k steps, far exceeding the 2,860-step budget. AdamW's weight decay was therefore essentially inactive (Wang and Aitchison, 2024). Re-evaluating AdamW with run-length-adjusted weight decay is required before concluding that matrix optimizers inherently dominate tuned AdamW at scale.
- **Confounded batch and decay schedules.** Doubling the batch late in training helped at 3B tokens but hurt at 1B tokens. This contrast is confounded by differences in decay schedules (linear 15% at 3B vs. 1−sqrt 45% at 1B) and prior training exposure. Furthermore, the 3B-token variants were evaluated on single seeds. Direct branching measurements (Merrill et al., 2025) are needed to isolate the true critical batch size across training.
- **Estimator biases in dynamics metrics.** Two-batch directional sharpness and the simple gradient noise scale were distorted by measurement artifacts: two-batch sharpness was dominated by a $1/\text{LR}$ step-noise bias, while the gradient noise scale reflected optimizer bouncing rather than optimization progress. Reliable curvature and noise diagnostics require probe-batch evaluations.

References

Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the Muon algorithm. *arXiv preprint arXiv:2505.16932*, 2025. URL https://arxiv.org/abs/2505.16932.

Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2103.00065.

Jeremy M. Cohen, Behrooz Ghorbani, Shankar Krishnan, Naman Agarwal, Sourabh Medapati, Michal Badura, Daniel Suo, David Cardoze, Zachary Nado, George E. Dahl, and Justin Gilmer. Adaptive gradient methods at the edge of stability. *arXiv preprint arXiv:2207.14484*, 2022. URL https://arxiv.org/abs/2207.14484.

Essential AI, Ishaan Shah, Anthony M. Polloreno, Karl Stratos, et al. Practical efficiency of Muon for pretraining. *arXiv preprint arXiv:2505.02222*, 2025. URL https://arxiv.org/abs/2505.02222.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL https://arxiv.org/abs/2407.21783.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2405.18392.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL https://arxiv.org/abs/2203.15556.

Shengding Hu, Yuge Tu, Xu Han, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2404.06395.

Ruixuan Huang, Hantao Huang, Yifan Huang, Ansheng You, Zhenxing Zhang, and Shuai Wang. Mechanism-driven monitors for preemptive detection of LLM training instability. *arXiv preprint arXiv:2606.28116*, 2026. URL https://arxiv.org/abs/2606.28116.

Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. Blog post, 2024. URL https://kellerjordan.github.io/posts/muon/.

Atli Kosson, Bettina Messmer, and Martin Jaggi. Rotational equilibrium: How weight decay balances learning across neural networks. In *International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2305.17212.

Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. NorMuon: Making Muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025. URL https://arxiv.org/abs/2510.05491.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2305.14342.

Jingyuan Liu, Jianlin Su, Xingcheng Yao, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025. URL https://arxiv.org/abs/2502.16982.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1711.05101.

Sadhika Malladi, Kaifeng Lyu, Abhishek Panigrahi, and Sanjeev Arora. On the SDEs and scaling rules for adaptive gradient algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.10287.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018. URL https://arxiv.org/abs/1812.06162.

William Merrill, Shane Arora, Dirk Groeneveld, and Hannaneh Hajishirzi. Critical batch size revisited: A simple empirical approach to large-batch language model training. *arXiv preprint arXiv:2505.23971*, 2025. URL https://arxiv.org/abs/2505.23971.

Alexandru Meterez, Depen Morwani, Jingfeng Wu, Costin-Andrei Oncescu, Cengiz Pehlevan, and Sham Kakade. Seesaw: Accelerating training by balancing learning rate and batch size scheduling. *arXiv preprint arXiv:2510.14717*, 2025. URL https://arxiv.org/abs/2510.14717.

muonium contributors. muonium: A distributed implementation of the Muon optimizer family for PyTorch. Software, 2025. URL https://github.com/tonyf/muonium.

Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *Proceedings of the 15th European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007.

Mark Rudelson and Roman Vershynin. Sampling from large matrices: An approach through geometric functional analysis. *Journal of the ACM*, 54(4), 2007. URL https://arxiv.org/abs/math/0503442.

Naoki Sato, Hiroki Naganuma, and Hideaki Iiduka. Convergence bound and critical batch size of Muon optimizer. *arXiv preprint arXiv:2507.01598*, 2025. URL https://arxiv.org/abs/2507.01598.

Tianqi Shen, Jinji Yang, Runze Shi, Jianhao Ma, Jiaye Teng, and Ziye Ma. Towards understanding the power and limits of the Muon optimizer: A river-valley perspective. *arXiv preprint arXiv:2606.21514*, 2026. URL https://arxiv.org/abs/2606.21514.

Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. Don't decay the learning rate, increase the batch size. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1711.00489.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing Shampoo using Adam. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2409.11321.

Xi Wang and Laurence Aitchison. How to set AdamW's weight decay as you scale model and dataset size. *arXiv preprint arXiv:2405.13698*, 2024. URL https://arxiv.org/abs/2405.13698.

Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. *arXiv preprint arXiv:2410.05192*, 2024. URL https://arxiv.org/abs/2410.05192.

Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025. URL https://arxiv.org/abs/2509.02046.

A W&B run names

- Entity: `bendoris0509-national-yang-ming-chiao-tung-university`
- Proxy runs: project `llama3.2-arch-small-scoring-model`
- Main-model runs: project `llama3.2-1b-optim-dynamics`
- Runs ending in `-s1` use seed 1; the others use the default seed (42).

Optimizer comparison (Sections 5–6)

Configuration	Run name(s)
Proxy model (174M)	
Proxy-1B NorMuon baseline	`proxy-dyn-normuon-1b-bs128`, `proxy-dyn-normuon-1b-bs128-s1`
Proxy-1B Muon	`proxy-dyn-muon-1b-bs128`, `-s1`
Proxy-1B Muon, LR 0.007 / AdamW 0.002	`proxy-dyn-muon0.007-adamw2e-3-1b-bs128`
Proxy-1B Muon, no norm WD	`proxy-dyn-muon-nonormwd-1b-bs128`, `-s1`
Proxy-1B SOAP	`proxy-dyn-soap-normwd-1b-bs128`, `-s1`
Proxy-1B SOAP, no norm WD	`proxy-dyn-soap-1b-bs128`, `-s1`
Proxy-1B SOAP, also preconditioning 1-D parameters	`proxy-dyn-soap-normwd-pc1d-1b-bs128`
Proxy-1B SOAP, LR 0.0028, no norm WD	`proxy-dyn-soap2.8e-3-1b-bs128`
Proxy-1B AdamW	`proxy-dyn-adamw-1b-bs128`
Proxy-1B Sophia (untuned / tuned)	`proxy-dyn-sophia-1b-bs128`, `proxy-dyn-sophia-tuned-1b-bs128`
Proxy-6B NorMuon	`proxy-dyn-normuon-6b-ckpt`, `proxy-dyn-normuon-6b-s1`
Proxy-6B Muon (with / without norm WD)	`proxy-dyn-muon-6b`, `proxy-dyn-muon-nonormwd-6b`
Proxy-6B SOAP (with / without norm WD)	`proxy-dyn-soap-normwd-6b`, `proxy-dyn-soap-no-normwd-6b`
Proxy-6B AdamW	`proxy-dyn-adamw-normwd-6b`, `proxy-dyn-adamw-normwd-6b-ckpt`, `proxy-dyn-adamw-6b`
Proxy-6B Sophia	`proxy-dyn-sophia-6b`, `proxy-dyn-sophia-tuned-normwd-6b`, `proxy-dyn-sophia-tuned-no-normwd-6b`
Main model (Llama-3.2-1B, Table 7)	
Main-1B NorMuon baseline	`normuon-default-1b-dyn`, `muon-default-1b-soapcmp`
Main-1B SOAP baseline	`soap3e-3-normwd-1b-dyn`, `soap3e-3-normwd-1b`
Main-1B AdamW	`adamw3e-4-1b-dyn`
Main-1B Sophia	`dedup-sophia-lr5e-5-1b`, `sophia-lr5e-5-decay0.1-1b`
Main-1B Muon	`muonium-muon-lr0.01-1b`
Main-3B NorMuon baseline	`normuon-default-3b-soapcmp`, `normuon-default-3b-dyn-gns-s1`
Main-3B SOAP baseline	`soap3e-3-normwd-3b`
Main-3B Sophia	`ramp-s05-lr5e-5-3b`

Main-model optimizer runs details (Table 7)

Optimizer / Budget	Run name (Job, W&B ID)	LR / Batch schedule	Pre-decay → Final (gain)
NorMuon 1B (run 1)	`normuon-default-1b-dyn` (570076, `i2eejmyt`)	0.01 + 3e-3, 32→64, 10% lin	3.0968 → 2.9324 (+0.164)
NorMuon 1B (repeat)	`muon-default-1b-soapcmp` (566172)	0.01 + 3e-3, 32→64, 10% lin	3.029 → 2.9454 (+0.084)
SOAP 1B (run 1)	`soap3e-3-normwd-1b-dyn` (570077, `s9br4vew`)	3e-3, norm wd, 32→64, 10% lin	3.1045 → 2.9545 (+0.150)
SOAP 1B (repeat)	`soap3e-3-normwd-1b` (566175)	3e-3, norm wd, 32→64, 10% lin	3.022 → 2.9522 (+0.070)
AdamW 1B	`adamw3e-4-1b-dyn` (570178, `hsgomr52`)	3e-4, norm wd, 32→64, 10% lin	3.2995 → 3.1886 (+0.111)
Sophia 1B	`dedup-sophia-lr5e-5-1b` (537295, `e9g8u50v`)	5e-5, $\rho = 0.05$, b512k, 10% lin	3.474 → 3.3710 (+0.103)
Muon 1B	`muonium-muon-lr0.01-1b` (`lbuf7ebv`)	0.01 + 3e-4, b512k, 10% lin	3.208 → 3.0630 (+0.145)
NorMuon 3B (run 1)	`normuon-default-3b-soapcmp` (566892, `k9k39bp7`)	0.01 + 3e-3, 32→64, 10% lin	2.9154 → 2.7075 (+0.208)
NorMuon 3B (seed 1)	`normuon-default-3b-dyn-gns-s1` (`5h5zj9mo`)	0.01 + 3e-3, 32→64, 10% lin	2.912 → 2.7053 (+0.207)
SOAP 3B	`soap3e-3-normwd-3b` (566891, `syx7kgpw`)	3e-3, norm wd, 32→64, 10% lin	2.8807 → 2.7014 (+0.179)
Sophia 3B	`ramp-s05-lr5e-5-3b` (540798, `um7ivu4h`)	5e-5, $\rho = 0.05$, 32→64, 10% lin	2.977 → 2.8790 (+0.098)

Normalized overshoot run details (Table 10)

Normalized overshoot per block is defined as $\mathrm{ov}_n = (D_1/\|g\|)/\cos(v,-g)$ computed from `D1`, `grad_norm`, and `upd_grad_align` (globally and under `per/<block>/...`). Values are medians over the stable-phase step range, with seed pairs averaged on the proxy.

Optimizer / Model	Run name (Job, W&B ID)	Stable window	Notes
Proxy model (174M, 1B tokens, batch 128)			
Muon (seed 0)	`proxy-dyn-muon-1b-bs128` (565026, `3mzsqwwg`)	Steps 1,030–3,433	Averaged with seed 1
Muon (seed 1)	`proxy-dyn-muon-1b-bs128-s1` (565032, `rbubsgpx`)	Steps 1,030–3,433	Averaged with seed 0
NorMuon (seed 0)	`proxy-dyn-normuon-1b-bs128` (566231, `8uspqif8`)	Steps 1,030–3,433	Averaged with seed 1
NorMuon (seed 1)	`proxy-dyn-normuon-1b-bs128-s1` (566904, `suvjvcjl`)	Steps 1,030–3,433	Averaged with seed 0
SOAP (seed 0)	`proxy-dyn-soap-normwd-1b-bs128` (565034, `j4igqpz4`)	Steps 1,030–3,433	Averaged with seed 1
SOAP (seed 1)	`proxy-dyn-soap-normwd-1b-bs128-s1` (565036, `xqthztfb`)	Steps 1,030–3,433	Averaged with seed 0
AdamW	`proxy-dyn-adamw-1b-bs128` (565027, `spzp9oen`)	Steps 1,030–3,433	Single seed
Sophia (tuned)	`proxy-dyn-sophia-tuned-1b-bs128` (565029, `fwbf2bne`)	Steps 1,030–3,433	$\rho = 0.05$, WD 0.2
Main model (Llama-3.2-1B, 1B tokens, batch 32→64)			
NorMuon	`normuon-default-1b-dyn` (570076, `i2eejmyt`)	Steps 500–1,900	Single seed
SOAP	`soap3e-3-normwd-1b-dyn` (570077, `s9br4vew`)	Steps 500–1,900	Single seed
AdamW	`adamw3e-4-1b-dyn` (570178, `hsgomr52`)	Steps 500–1,900	Single seed
Muon / Sophia	–	–	No per-block runs logged

Norm-layer dynamics run details (Table 13)

Relative update size $\|\Delta W\|/\|W\|$ is the stable-phase median of `per/norm/upd_rel`. Weight norm shrinkage reports Frobenius norm `per/norm/w_norm` from initialization ($S = 0$) to the end of training.

Optimizer / Model	Run name (Job, W&B ID)	Stable window	Norm shrinkage ($\|W\|_F$)
Proxy model (174M, 6B tokens, batch 256)			
Muon	`proxy-dyn-muon-6b` (561805, `qo1uufmc`)	Steps 3,090–10,300	138.3 → 56.9
NorMuon	`proxy-dyn-normuon-6b-ckpt` (567242, `89gb67q3`)	Steps 3,090–10,300	138.3 → 56.7
SOAP	`proxy-dyn-soap-normwd-6b` (563152, `o23plv5o`)	Steps 3,090–10,300	138.4 → 36.3
AdamW	`proxy-dyn-adamw-normwd-6b` (563002, `5jg6wz7p`)	Steps 3,090–10,300	138.5 → 70.4
Sophia	`proxy-dyn-sophia-6b` (561908, `0n0xcyk3`)	Steps 3,090–10,300	138.6 → 80.0
Main model (Llama-3.2-1B, 1B tokens, batch 32→64)			
NorMuon	`normuon-default-1b-dyn` (570076, `i2eejmyt`)	Steps 500–1,900	258.1 → 139.2
SOAP	`soap3e-3-normwd-1b-dyn` (570077, `s9br4vew`)	Steps 500–1,900	257.9 → 85.1
AdamW	`adamw3e-4-1b-dyn` (570178, `hsgomr52`)	Steps 500–1,900	259.8 → 241.1
Muon / Sophia	–	–	No norm dynamics logged

Per-block changes (Section 7)

Variant	Run name(s)
Q, K, V per head	`proxy-dyn-normuon-splitqkv-1b-bs128`, `-s1`
K, V per head	`proxy-dyn-normuon-splitkv-1b-bs128`
K, V LR $\times 0.5$	`proxy-dyn-normuon-kv0.5-1b-bs128`
SOAP embedding	`proxy-dyn-normuon-soapembed-1b-bs128-v2` (v1 invalid: constant-LR bug)
Norm-gain LR 0.001	`proxy-dyn-normuon-normlr1e-3-1b-bs128`

Optimizer-switch branches (Section 8)

Configuration	Run name(s)
NorMuon full run and branches	`proxy-dyn-normuon-6b-ckpt`, `...-ckpt-branch3000`, `-branch5500`, `-branch8000`
AdamW full run and branches	`proxy-dyn-adamw-normwd-6b-ckpt`, `...-ckpt-branch3000`, `-branch5500`, `-branch8000`

Decay length and shape (Section 9)

Configuration	Run name(s)
NorMuon 30% linear	`normuon-decay0.3-1b-dyn`
NorMuon 30% 1−sqrt	`normuon-decay0.3-1sqrt-1b-dyn`, `normuon-decay0.3-1sqrt-1b-dyn-gns-s1`
SOAP 30% 1−sqrt	`soap3e-3-decay0.3-1sqrt-1b-dyn`
AdamW 30% linear	`adamw3e-4-decay0.3-1b-dyn`

Batch schedule, 1B tokens (Section 10)

Variant	Run name(s)
Baseline	`normuon-decay0.3-1sqrt-1b-dyn`, `normuon-decay0.3-1sqrt-1b-dyn-gns-s1`
LR $\times\sqrt{2}$ at the batch increase	`normuon-decay0.3-1sqrt-ramplr1.41-1b-dyn-gns`, `-s1`
Second doubling to 128, decay from 0.55B	`normuon-decay0.192-1sqrt-ramp128-1b-dyn-gns`, `-s1`
Second doubling to 128, decay from 0.455B	`normuon-decay0.3-1sqrt-ramp128-1b-dyn-gns`, `-s1`

Batch schedule, 3B tokens (Section 10)

Label	Run name	Resumed from
A	`normuon-default-3b-dyn-gns-s1`	–
B	`normuon-ramp64e-3b-dyn-gns-s1`	–
C	`normuon-ramp128-decay64-3b-dyn-gns-s1`	B, checkpoint-3432 (1.5B tokens)
D	`normuon-ramp128-b-3b-dyn-gns-s1`	–
E	`normuon-ramp128-decay256-3b-dyn-gns-s1`	C, checkpoint-4434 (2.55B tokens)
F	`normuon-ramp128at1b-3b-dyn-gns-s1` (running)	B, checkpoint-2479 (1.0B tokens)
–	`normuon-ramp64e-decay32-3b-dyn-gns-s1` (crashed)	B, checkpoint-5436

Data files.

- `wandb_export.json`: 56 runs, history down-sampled to at most 150 points per series.
- `gns_export.json`: the 14 main-model runs that log the noise scale, with $|G|^2$ and $\mathrm{tr}\Sigma$ averaged in token windows, plus eval and LR curves.
- Dynamics keys in W&B follow the pattern `dyn_<metric>/<block>`. For example, the noise-scale keys are `dyn_gns_G2/<block>`, `dyn_gns_trS/<block>` and `dyn_gns_B/<block>`.

B How the settings were tuned

The sweeps were run before this study and are recorded in `experiments.md`. Table 23 summarizes them; Appendix C gives the full proxy sweep results.

Table 23: How each optimizer setting was tuned. "Grid edge" means the best value was the largest or smallest one tried, so the true optimum may lie outside the grid.

Optimizer	Setting	How it was tuned
Main model (Llama-3.2-1B)		
NorMuon + AdamW	0.01 + 0.003	NorMuon matched Muon at 0.01 and was flat over 0.0075–0.01. AdamW part swept over 3×10^{-4}, 10^{-3}, 3×10^{-3}, 6×10^{-3}, 10^{-2} with the batch ramp: $3\times10^{-3}\approx6\times10^{-3}$ best.
Muon + AdamW	0.01 + 3×10^{-4}	Muon LR 0.005 / 0.01 / 0.02 / 0.04 at 1B tokens; the AdamW part was retuned to 0.003 only for NorMuon.
SOAP	0.003	7.5e-4 / 1.5e-3 / 3e-3 at 1B tokens. Best at the grid edge; 6e-3 untested.
AdamW	3×10^{-4}	Only a 2M-token-batch LR scan (238 steps, best at the grid edge, noisy); the batch sweep used $\sqrt{\text{batch}}$ scaling. **Weakly tuned.**
Sophia	5×10^{-5}, $\rho=0.05$	0.5M batch, 1.1B tokens. 5×10^{-5} matched 10^{-4} but was more consistent across seeds; 2×10^{-4} is too high.
Proxy (174M)		
NorMuon + AdamW	0.01 + 0.003	Not swept on the proxy; copied from the main model. Only check: 0.7× both LRs at batch 128 tied.
Muon + AdamW	0.01 + 0.003	Muon 0.005–0.04 and AdamW part 3×10^{-4}–10^{-2}; the same optimum as the main model.
SOAP	0.004	1e-3 to 8e-3 at 1B tokens, batch 256. At batch 128, 2.8e-3 beat 4e-3 (3.239 vs. 3.248), so 0.004 is slightly high for the 1B-token runs.
AdamW	0.001	5e-4 to 6e-3 at 1B tokens, batch 256; 1e-3 and 2e-3 tied.
Sophia	2×10^{-4}, $\rho=0.05$, WD 0.2	Retuned for the target clipping ("win") rate at 2B tokens (LR $\times\rho$ grid). The old setting was 2×10^{-4}, $\rho=0.01$, WD 0.1.

What is held fixed.

- Within each model, the optimizer hyperparameters are fixed across all experiments in this report. The exceptions are the deliberate variants and the inconsistencies in Table 24.
- The proxy and main model share NorMuon's LRs.
- SOAP and AdamW use different LRs on the two models (proxy 0.004 / 0.001, main 0.003 / 3×10^{-4}), each from its own sweep.

Table 24: Every departure from the fixed optimizer settings in the reported runs.

Departure	Runs affected
Deliberate variants	
Norm-gain WD off	Muon and SOAP "no norm WD" variants
Muon LR 0.007 / AdamW part 0.002	one proxy-1B run (0.7× LR check)
SOAP LR 0.0028, norm WD off	one proxy-1B run (0.7× LR check)
SOAP also preconditions 1-D parameters	one proxy-1B run
Sophia old vs. retuned	old: $\rho = 0.01$, WD 0.1; retuned: $\rho = 0.05$, WD 0.2
Per-block NorMuon changes	Section 7
LR $\times\sqrt{2}$ at the batch increase	main model, Section 10
Inconsistencies to keep in mind	
Norm WD off in the proxy-1B AdamW and Sophia runs, and in the proxy-6B AdamW (one of two) and old-Sophia runs	These runs use the old default (off), while NorMuon, Muon and SOAP have it on. At proxy-1B norm WD is worth ≈0.004–0.012 (Section 5), small next to the AdamW and Sophia gaps.
AdamW ε on the main model	These runs used $\varepsilon = 10^{-8}$ (logged), whereas the main-model AdamW setting was tuned with $\varepsilon = 10^{-5}$. AdamW's LR on the main model is also only weakly tuned, so its main-model results are a weak baseline.
Main-model Muon and Sophia batch schedules	Muon 1B and Sophia 1B used fixed 0.5M token batch (1,907 steps) from earlier sweeps rather than the 32→64 batch ramp (2,860 steps). Sophia 3B used the 32→64 ramp (7,628 steps) evaluated on doc-level loss (2.8790).

C Hyperparameter sweeps that set the baselines

These runs are in the proxy project and predate the instrumented study. All use the proxy model at batch 256 sequences, 10% linear decay and norm WD off. Values are final eval loss; bold marks the value adopted as the baseline.

Optimizer	Budget, fixed settings	Swept values → final eval loss
Muon	1B tokens; AdamW group 0.001	Muon LR 0.005 → 3.2846; **0.01** → 3.2686; 0.02 → 3.2829; 0.04 → 3.4418
Muon	1B tokens; Muon LR 0.01	AdamW-group LR 3×10^{-4} → 3.3465; 0.001 → 3.2686; **0.003** → 3.2521; 0.01 → 3.2556
SOAP	1B tokens	LR 0.001 → 3.2777; 0.002 → 3.2500; **0.004** → 3.2469; 0.006 → 3.2603; 0.008 → 3.2734
AdamW	1B tokens	LR 5×10^{-4} → 3.5291; **0.001** → 3.4057; 0.002 → 3.4062; 0.003 → 3.4285; 0.006 → 6.5220 (diverged)
Sophia	1B tokens; $\rho = 0.05$, WD 0.1	LR 5×10^{-5} → 3.8512; 10^{-4} → 3.6165; 2×10^{-4} → 3.6653; 3×10^{-4} → 3.6608; $\geq 6 \times 10^{-4}$ diverged
Sophia	1B tokens; LR 10^{-4}, WD 0.1	ρ 0.003 → 3.5815; 0.01 → 3.5993; 0.05 → 3.6165; 0.2 → 3.7977
Sophia	2B tokens; WD 0.2	grid of LR $(1\text{–}6) \times 10^{-4}$ and $\rho \in \{0.02, 0.03, 0.05, 0.1\}$; best **LR** 2×10^{-4}**,** $\rho = 0.05$ → 3.2755 (adopted as "Sophia tuned")

Run-name patterns:

- `scorer-muon<LR>-adamw<LR>-1b`
- `proxy-soap-lr<LR>-1b`, `proxy-adamw-lr<LR>-1b`

- `proxy-sophia-lr<LR>[-rho<rho>][-wd0.2-2b]`

D Logger validation

`test_optim_dynamics.py` checks the logger on problems with known answers; all checks pass.

- **Alignment:** plain SGD gives alignment $= 1$.
- **Sharpness:** on a quadratic, directional sharpness is 1.211744 vs. 1.211747 exact.
- **Spectra:** power-iteration spectral norms match an exact SVD.
- **Effective rank:** an orthogonal matrix gives 16 (full), a rank-1 matrix gives 1, and singular values $[2, 1]$ give 1.6494, all exact.
- $1/$**LR bias:** the two-batch sharpness is 6.3 / 20.8 / 75.6 at LR 0.2 / 0.05 / 0.0125, while the probe-batch sharpness stays ≈ 2.5.
- **Probe metrics:** exact on a single step; training is bit-identical with and without the probe.
- **Noise scale:** estimate 11.15 vs. exact 11.12.